



**Diseño e implementación de la extensión 'C'
(instrucciones comprimidas) de
RISC-V en un procesador pipelined**

Integrantes:

APELLIDOS, NOMBRES	Participación
Thiago Gabriel Rivera Matta	100%
Jesús Valentín Niño Castañeda	100%
Reyes Medina, Jordinn Martin	100%

ASIGNATURA:

Arquitectura de Computadoras

DOCENTE:

Carlos Williams

ÍNDICE

1. Implementación de las Instrucciones.....	3
1.1 Instrucciones Implementadas.....	3
1.2 Cambios del Código (Mismos que Parte 1).....	4
1.2.1 El Módulo Descompresor (decompressor.v).....	4
1.2.2 Adaptación del Datapath y Control del PC (datapath.v).....	5
1.2.3 Fetch Desalineado en la Memoria de Instrucciones (imem.v).....	5
1.2.4 Limpieza y Bloques de Ejecución (aludec.v y pipereg.v).....	6
1.2.5 Monitor Universal de Pruebas (testbench.v).....	6
1.3 Explicación de las Instrucciones.....	7
c.lw y c.sw.....	7
c.lwsp y c.swsp.....	7
c.beqz y c.bnez.....	8
c.j y c.jal.....	9
c.jr y c.jalr.....	9
2. Resultados.....	10
2.1 Programas ISA.....	10
Prueba 1: Operaciones de Memoria en la Pila (c.swsp y c.lwsp).....	10
Prueba 2: Operaciones de Memoria con Base y Desplazamiento (c.sw y c.lw)..	11
Prueba 3: Saltos Incondicionales y Desalineamiento (c.jal, c.jr, c.j y c.jalr).....	11
Prueba 4: Saltos Condicionales y Riesgos de Control (c.beqz y c.bnez).....	12
2.2 Validación de cada una de las instrucciones.....	13
Validación de c.swsp y c.lwsp:.....	13
Validación de c.sw y c.lw:.....	14
Validación de jumps(c.jal, c.jr, c.j y c.jalr):.....	15
Validación de branches(c.beqz y c.bnez):.....	15
3. Algoritmo.....	16
3.1 Programa ISA: QuickSort.....	16
RV32I:.....	16
RV32C:.....	19
3.2 Waveforms.....	23
RV32I:.....	23
RV32C:.....	24
3.3 Comparativa.....	25
Análisis de Resultados:.....	26
4. Bibliografía.....	27

1. Implementación de las Instrucciones

1.1 Instrucciones Implementadas

La implementación de la extensión 'C' incluye un total de 20 instrucciones comprimidas. A continuación, se presenta la tabla de *Encoding* unificada que detalla la sección (**op**), los bits de función (**funct3**) y el formato RVC de **todas las instrucciones de la Parte 1 y Parte 2**, así como su equivalente en el ISA base de 32 bits.

Instrucción	Cuadrante (op)	funct3	Formato RVC	Operación / Equivalente RV32I
Parte 1 (Aritmética y Lógica)				
c.addi	01	000	CI	Suma de inmediato (addi)
c.li	01	010	CI	Carga de inmediato (addi)
c.lui	01	011	CI	Carga de parte alta (lui)
c.srli / c.srai	01	100	CB	Corrimientos lógicos/aritmét icos
c.andi	01	100	CB	AND lógico con inmediato
c.sub / c.xor c.or / c.and	01	100	CA	Operaciones aritmético/lógi cas entre registros restringidos
c.slli	10	000	CI	Corrimiento izquierdo (slli)

c.add / c.mv	10	100	CR	Suma y copia de registros genéricos
Parte 2 (Memoria y Flujo)				
c.lw	00	010	CL	Carga de memoria c/ offset
c.sw	00	110	CS	Escritura en memoria c/ offset
c.lwsp	10	010	CI	Carga relativa al Stack Pointer
c.swsp	10	110	CSS	Escritura relativa al Stack Pointer
c.beqz	01	110	CB	Salto si es cero (beq rs1, x0)
c.bnez	01	111	CB	Salto si no es cero (bne rs1, x0)
c.j	01	101	CJ	Salto incondicional (jal x0)
c.jal	01	001	CJ	Salto incondicional y enlace (jal x1)
c.jr	10	100	CR	Salto a registro (jalr x0, rs1)

c.jalr	10	100	CR	Salto a registro y enlace (jalr x1, rs1)
--------	----	-----	----	--

1.2 Cambios del Código

1.2.1 El Módulo Descompresor (decompressor.v)

El corazón de la extensión 'C' es un traductor combinacional que toma instrucciones de 16 bits y las expande a sus equivalentes de 32 bits del ISA base de RISC-V en tiempo real. Se creó un módulo completamente nuevo (decompressor.v) que lee los bits menos significativos (op) y los bits de función (funct3) para determinar a qué cuadrante pertenece la instrucción comprimida y mapear los campos (como los registros restringidos x8-x15) a la codificación estándar de 32 bits.

```
module decompressor(
    input wire [15:0] instr_c,
    output reg [31:0] instr_32
);
    wire [1:0] op = instr_c[1:0];
    wire [2:0] funct3 = instr_c[15:13];

    // Expansión de registros comprimidos x8-x15
    wire [4:0] rs1_c = {2'b01, instr_c[9:7]};
    wire [4:0] rs2_c = {2'b01, instr_c[4:2]};

    always @* begin
        case (op)
            2'b00: begin /* Cuadrante 0: c.lw, c.sw */ end
            2'b01: begin /* Cuadrante 1: c.addi, c.jal, c.beqz, etc. */ end
            2'b10: begin /* Cuadrante 2: c.slli, c.lwsp, c.add, etc. */ end
            default: instr_32 = 32'h00000013; // NOP de seguridad
        endcase
    end
endmodule
```

1.2.2 Adaptación del Datapath y Control del PC (datapath.v)

En el datapath original, el Program Counter (PC) siempre avanzaba sumando 4 bytes (PC + 4). Con la extensión 'C', el salto del PC ahora es dinámico: debe sumar 2 si la instrucción actual es de 16 bits, o 4 si es de 32 bits. Además, se insertó el descompresor en la etapa de Fetch para que evalúe la instrucción entrante. Si la instrucción detectada es comprimida

(evaluando si los últimos dos bits son distintos de 11), se pasa al registro de segmentación (IF/ID) la versión descomprimida; de lo contrario, pasa la instrucción original.

```
// Detección de instrucción comprimida
wire is_compressed = (InstrF[1:0] != 2'b11);

// Instanciación del descompresor
wire [31:0] InstrF_decomp;
decompressor decomp (.instr_c(InstrF[15:0]), .instr_32(InstrF_decomp));

// Selección de la instrucción final que pasará al pipeline
wire [31:0] InstrF_final = is_compressed ? InstrF_decomp : InstrF;

// Cálculo dinámico del salto del Program Counter
wire [31:0] PCStep = is_compressed ? 32'd2 : 32'd4;
adder pcaddstep(.a(PCF), .b(PCStep), .y(PCPlusStepF));

// Actualización del registro IF/ID con la instrucción final y el PCStep
pipereg r_if_id_instr (.clk(clk), .reset(reset), .en(~StallD), .clr(PCSrcE),
.d(InstrF_final), .q(InstrD));
```

1.2.3 Fetch Desalineado en la Memoria de Instrucciones (imem.v)

Dado que las instrucciones comprimidas ocupan 2 bytes (16 bits), el PC puede terminar apuntando a direcciones que terminan en 2 (por ejemplo, 0x...002, 0x...006), lo cual rompe el alineamiento estándar de palabras de 32 bits. Si una instrucción de 32 bits empieza en un límite de "half-word" desalineado, cruzará la frontera de la palabra de memoria actual. Para solucionarlo, se modificó la memoria de instrucciones para que concatene los 16 bits superiores de la palabra actual con los 16 bits inferiores de la siguiente palabra cuando detecte que la dirección está desalineada ($a[1] == 1$).

```
wire [29:0] word_addr = a[31:2];
wire half_aligned = a[1]; // Detecta si termina en 2 (ej. 0x02)

// Si está desalineado, une la mitad superior de esta palabra con la mitad
inferior de la siguiente
assign rd = half_aligned ? {RAM[word_addr + 1][15:0],
RAM[word_addr][31:16]} : RAM[word_addr];
```

1.2.4 Limpieza y Bloques de Ejecución (aludec.v y pipereg.v)

Para evitar condiciones de carrera ("latches" inferidos) y asegurar un diseño combinacional y secuencial limpio, se refactorizó la lógica en varios archivos envolviendo sentencias en bloques begin ... end. Aunque esto no cambia la arquitectura a nivel macro, garantiza que la síntesis en hardware asigne las señales correctamente. Asimismo, los registros de pipeline (pipereg.v) se limpiaron para manejar el en (Stall) y clr (Flush) de una forma estructurada

con prioridad, asegurando que si ocurre un Stall (en == 0), el registro simplemente mantenga su valor:

```
always @* begin
    case(ALUOp)
        // ...
        2'b10: begin // R-type e I-type ALU
            case(func3)
                3'b000: begin
                    if (func7b5 & opb5) ALUControl = 4'b0001;
                    else ALUControl = 4'b0000;
                end
            // ...
            endcase
        end
        // ...
    endcase
end
```

1.2.5 Monitor Universal de Pruebas (testbench.v)

Finalmente, para validar programas dinámicos que combinan instrucciones ISA y RVC, el testbench se ajustó. Ahora no busca un resultado en una dirección arbitraria sujeta a cambios por la compresión, sino que establece un "STOP UNIVERSAL": se asume que todos los algoritmos de prueba terminarán escribiendo su resultado final en la dirección de memoria 200. Esto permite testear quicksort_rv32c, matrix_rv32c y tree_rv32c sin modificar el testbench cada vez.

```
// Monitor Universal de Resultados
always @(negedge clk) begin
    if(MemWrite) begin
        $display("RAM Write: Escribio %d en la direccion %d", WriteData,
DataAdr);

        // CONDICIÓN UNIVERSAL: Termina al escribir en la dir 200
        if(DataAdr === 200) begin
            $display("¡EXITO TOTAL! El programa ha finalizado.");
            $finish;
        end
    end
end
end
```

1.2.6 Cambios Extras y Compatibilidad en la Hazard Unit (Parte 2)

Aunque el diseño principal del datapath (PC dinámico y Fetch desalineado) se estableció en la primera fase, la Parte 2 introduce instrucciones críticas: saltos de control de flujo (**jumps/branches**) e instrucciones de memoria. El diseño propuesto soluciona los conflictos de forma nativa sin añadir hardware redundante.

Al inyectar el código de 32 bits expandido directamente desde el `decompressor.v` en la etapa de Fetch, la `Hazard Unit` original del ISA base es capaz de detectar los riesgos nuevos de la Parte 2, tales como riesgos de datos (*Load-Use Hazards* con `c.lwsp`) y riesgos de control (*Control Hazards* con `c.beqz` o `c.jal`), emitiendo las señales `StallF`, `StallD` y `FlushE` con total normalidad frente a instrucciones comprimidas, limpiando el cauce de la misma forma en que lo haría con instrucciones de 32 bits.

1.3 Explicación de las Instrucciones

A continuación se detalla la lógica de decodificación de cada instrucción implementada. Los comentarios dentro del código explican de forma explícita cómo se expanden los campos comprimidos para reconstruir la estructura estándar de 32 bits del ISA RISC-V.

`c.lw` y `c.sw`

Estas instrucciones pertenecen al cuadrante 2'b00 y gestionan el acceso a la memoria de datos utilizando los registros restringidos x8 a x15. En ambas, la dirección base se lee de `rs1_c` y el dato se lee o guarda en `rs2_c`. El descompresor expande automáticamente estas direcciones de 3 bits a 5 bits anteponiendo 2'b01.

- `c.lw` (`funct3 = 3'b010`): Se expande al formato I de 32 bits (`lw`). El inmediato de 12 bits se construye tomando los bits 5, 12:10, y 6 de la instrucción comprimida, rellenando con ceros los bits superiores y desplazando dos posiciones a la izquierda (2'b00) ya que se trata de alineamiento de palabras (4 bytes). El opcode se establece en 7'b0000011 (Load) y `funct3` en 3'b010.
- `c.sw` (`funct3 = 3'b110`): Se expande al formato S de 32 bits (`sw`). A diferencia del `lw`, el inmediato en el formato S está dividido. El descompresor coloca la parte alta del offset (`instr_c[5]`, `instr_c[12]`) en los bits 31:25 y la parte baja (`instr_c[11:10]`, `instr_c[6]`, 2'b00) en los bits 11:7. El opcode es 7'b0100011 (Store).

None

```
always @* begin
    case (op)
        2'b00: begin
            case (funct3)
                3'b010: instr_32 = {{5'b0, instr_c[5], instr_c[12:10],
instr_c[6], 2'b00}, rs1_c, 3'b010, rs2_c, 7'b0000011}; // c.lw
                3'b110: instr_32 = {{5'b0, instr_c[5], instr_c[12]}, rs2_c,
rs1_c, 3'b010, {instr_c[11:10], instr_c[6], 2'b00}, 7'b0100011}; // c.sw
                default: instr_32 = 32'h00000013;
            endcase
        end
    end
    // ...
```


c.lwsp y c.swsp

Ubicadas en el cuadrante 2'b10, estas instrucciones están altamente optimizadas para acceder a la pila (Stack) utilizando implícitamente el Stack Pointer (sp o registro x2). Operan sobre cualquier registro de la arquitectura (x0-x31).

- c.lwsp (funct3 = 3'b010): Se mapea a la instrucción lw (Formato I). El registro base (rs1) se fija en hardware a 5'b00010 (sp). El registro destino (rd) se extrae de instr_c[11:7]. El inmediato se reorganiza desde los bits dispersos de la instrucción y termina con 2'b00 para garantizar alineación de palabra.
- c.swsp (funct3 = 3'b110): Se mapea a la instrucción sw (Formato S). Al igual que en c.lwsp, el registro base (rs1) se fija en 5'b00010. El registro fuente (rs2) se toma de instr_c[6:2]. El inmediato se divide según el formato S, ubicando los bits correspondientes en las secciones altas (31:25) y bajas (11:7) de la instrucción de 32 bits.

```
None
2'b10: begin
    case (funct3)
        // ...
        3'b010: instr_32 = {{4'b0, instr_c[3:2], instr_c[12],
instr_c[6:4], 2'b00}, 5'b00010, 3'b010, instr_c[11:7], 7'b0000011}; //
c.lwsp
        // ...
        3'b110: instr_32 = {{4'b0, instr_c[8:7], instr_c[12]},
instr_c[6:2], 5'b00010, 3'b010, {instr_c[11:9], 2'b00}, 7'b0100011}; //
c.swsp
        // ...
```

c.beqz y c.bnez

Ambas instrucciones de salto condicional se ubican en el cuadrante 2'b01. Estas instrucciones validan una condición contra el valor cero, por lo que el hardware asigna implícitamente el registro constante x0 (5'b00000) como segundo operando (rs2). Operan sobre el conjunto de registros restringidos rs1_c.

- c.beqz (funct3 = 3'b110): Se expande a beq (Formato B). El inmediato se reconstruye tomando en cuenta la extensión de signo (instr_c[12]) y reensamblando el target offset. El registro a evaluar se conecta a rs1_c, mientras que rs2 es forzado a cero. El opcode es 7'b1100011 y funct3 es 3'b000.
- c.bnez (funct3 = 3'b111): Se comporta exactamente igual que la instrucción anterior, pero mapea a bne asignando 3'b001 en el campo funct3.

```
None
2'b01: begin
    case (funct3)
        // ...
```

```

        3'b110: instr_32 = {{{4{instr_c[12]}}}, instr_c[6:5],
instr_c[2]}, 5'b00000, rs1_c, 3'b000, {instr_c[11:10], instr_c[4:3],
instr_c[12]}, 7'b1100011}; // c.beqz
        3'b111: instr_32 = {{{4{instr_c[12]}}}, instr_c[6:5],
instr_c[2]}, 5'b00000, rs1_c, 3'b001, {instr_c[11:10], instr_c[4:3],
instr_c[12]}, 7'b1100011}; // c.bnez
        // ...

```

c.j y c.jal

En el cuadrante 2'b01, los saltos incondicionales relativos al PC (c.j y c.jal) comparten la misma lógica de construcción de inmediato, pero difieren en el manejo del registro de enlace (link register).

La traducción ensambla un inmediato tipo J (20 bits) mediante una concatenación compleja que reordena los bits revueltos de la instrucción de 16 bits (instr_c[12], instr_c[8], instr_c[10:9], etc.) y aplica extensión de signo.

- c.j (funct3 = 3'b101): Mapea a jal x0, offset. Descarta la dirección de retorno forzando el registro destino a 5'b00000 (x0).
- c.jal (funct3 = 3'b001): Mapea a jal x1, offset. Guarda la dirección de retorno forzando el registro destino a 5'b00001 (ra o x1).

```

None
2'b01: begin
    case (funct3)
        // ...
        3'b001, 3'b101: // c.jal / c.j
            instr_32 = {instr_c[12], instr_c[8], instr_c[10:9],
instr_c[6], instr_c[7], instr_c[2], instr_c[11], instr_c[5:3], instr_c[12],
{8{instr_c[12]}}}, (funct3==3'b001 ? 5'b00001 : 5'b00000), 7'b1101111};
        // ...

```

c.jr y c.jalr

Finalmente, los saltos incondicionales a registro se ubican en el cuadrante 2'b10 bajo el mismo funct3 = 3'b100. Para distinguirlos de otras operaciones (como c.mv o c.add), el descompresor exige que los bits instr_c[6:2] sean explícitamente 0.

Ambas se traducen a la instrucción de 32 bits jalr (Formato I) estableciendo el inmediato en 12'b0, ya que en la versión comprimida no hay offset. La diferenciación radica en el bit 12:

- c.jr (instr_c[12] == 0): Al no ser una llamada a subrutina, la dirección de retorno no se guarda. El registro destino se cablea a 5'b00000 (x0).
- c.jalr (instr_c[12] == 1): Al ejecutarse un salto con enlace, el registro destino se cablea a 5'b00001 (ra o x1) para almacenar automáticamente el PC de retorno. El registro objetivo de salto (rs1) siempre se extrae de instr_c[11:7].

```

None
2'b10: begin
    case (funct3)
        // ...
        3'b100: begin
            if (instr_c[12] == 0) begin
                if (instr_c[6:2] == 0) // c.jr rs1
                    instr_32 = {12'b0, instr_c[11:7], 3'b000,
5'b00000, 7'b1100111};
                // ...
            end else begin
                if (instr_c[6:2] == 0) // c.jalr rs1
                    instr_32 = {12'b0, instr_c[11:7], 3'b000,
5'b00001, 7'b1100111};
                // ...
            end
        end
    endcase
end

```

2. Resultados

2.1 Programas ISA

Para evaluar la correcta implementación de las nuevas características y el comportamiento del hardware en esta entrega, se han diseñado cuatro programas de prueba. La característica principal de estos códigos es someter a la arquitectura a escenarios de estrés específicos, tales como desalineamiento del Program Counter (PC), riesgos de control (hazards) y la expansión de registros implícitos. A continuación se detalla explícitamente el funcionamiento de cada prueba.

Prueba 1: Operaciones de Memoria en la Pila (c.swsp y c.lwsp)

Valida las instrucciones de acceso a memoria que asumen implícitamente el uso del Stack Pointer (sp o x2). Además, verifica la correcta activación de la unidad de adelantamiento (Forwarding Unit) cuando el resultado de una carga de memoria es requerido inmediatamente por la siguiente instrucción aritmética.

```

None
# Código Ensamblador
addi sp, x0, 256      # Configura el stack pointer en la dirección de
                      # memoria 256
addi a0, x0, 42       # Carga el dato '42' en a0 (x10)
c.swsp a0, 0(sp)      # Guarda a0 en mem[sp]. (Expansión: sw a0, 0(sp))
c.lwsp a1, 0(sp)      # Carga desde mem[sp] a a1 (x11). (Expansión: lw a1,
                      # 0(sp))
add a2, a0, a1        # a2 = 84. Activa el Forwarding Unit (MEM/WB -> EX)

```

```
# Mapa de Memoria (Hexadecimal / Binario)
10000113 // 0x00: addi sp, x0, 256 (32-bit)
02A00513 // 0x04: addi a0, x0, 42 (32-bit)
4582C02A // 0x08: c.lwsp a1, 0(sp) [4582] | c.swsp a0, 0(sp) [C02A]
(Empaquetadas)
00B50633 // 0x0C: add a2, a0, a1 (32-bit)
```

Objetivo en el Waveform: Demostrar que el descompresor expande correctamente estas instrucciones inyectando físicamente $rs1 = x2$. Se debe observar nítidamente cómo el PC avanza +4 en las instrucciones estándar de 32 bits (addi) y +2 tras procesar las instrucciones comprimidas de 16 bits (c.swsp/c.lwsp).

Prueba 2: Operaciones de Memoria con Base y Desplazamiento (c.sw y c.lw)

Evalúa las instrucciones comprimidas de memoria estándar que operan sobre el conjunto de registros restringidos (x8 a x15 en la arquitectura RISC-V).

```
None
# Código Ensamblador
addi a0, x0, 100 # Puntero base en a0 (x10, registro válido para RVC)
addi a1, x0, 77 # Dato a guardar en a1 (x11)
c.sw a1, 0(a0) # Guarda el valor de a1 en mem[a0]
c.lw a2, 0(a0) # Carga el valor desde mem[a0] hacia a2 (x12)
sub a3, a2, a1 # a3 = 0. Verifica correcta lectura pos-compresión

# Mapa de Memoria (Hexadecimal / Binario)
06400513 // 0x00: addi a0, x0, 100 (32-bit)
04D00593 // 0x04: addi a1, x0, 77 (32-bit)
4210C10C // 0x08: c.lw a2, 0(a0) [4210] | c.sw a1, 0(a0) [C10C]
(Empaquetadas)
40B606B3 // 0x0C: sub a3, a2, a1 (32-bit)
```

Objetivo en el Waveform: Confirmar que el decodificador mapea de forma exacta los bits reducidos hacia su equivalente en la arquitectura base (ej. 010 se expande a x10). Asimismo, vigilar que el procesador sea capaz de leer correctamente una instrucción de 32 bits de forma inmediata tras procesar un bloque donde finalizó una instrucción de 16 bits.

Prueba 3: Saltos Incondicionales y Desalineamiento (c.jal, c.jr, c.j y c.jalr)

Comprueba el flujo de control mediante saltos incondicionales (relativos al PC y directos a registro). Permite observar el almacenamiento de la dirección de retorno en el Link Register (ra o x1).

```

None
# Código Ensamblador
main:
    c.jal subrutina    # Salta a 'subrutina', guarda el PC+2 de retorno en ra
(x1)
    c.j fin           # Salto incondicional (evita la subrutina si falla el
c.jr)

subrutina:
    addi a0, x0, 5     # Operación de prueba dentro de la subrutina (a0 = 5)
    c.jr ra            # Retorna al valor guardado en 'ra' (apunta a c.j fin)

fin:
    addi a1, a0, 5     # a1 = 10 (Se ejecuta tras el salto final)

# Mapa de Memoria (Hexadecimal / Binario)
A0212011 // 0x00: c.j fin [A021] | c.jal subrutina [2011]
00500513 // 0x04: addi a0, x0, 5 (32-bit)
05938082 // 0x08: Mitad inferior de 'addi a1, a0, 5' [0593] | c.jr ra
[8082]
00010055 // 0x0C: c.nop (Relleno) [0001] | Mitad superior de 'addi a1, a0,
5' [0055]

```

Objetivo en el Waveform: Mostrar claramente el vaciado (Flush) en el pipeline derivado del salto ejecutado. Es imperativo observar que el PC transita hacia direcciones desalineadas (múltiplos de medias palabras o 2 bytes) sin generar fallos en el datapath.

Prueba 4: Saltos Condicionales y Riesgos de Control (c.beqz y c.bnez)

Verifica el comportamiento de la evaluación lógica exclusiva contra el valor cero. La estructura de la prueba inyecta deliberadamente instrucciones que deben ser anuladas en el pipeline si el salto se toma exitosamente.

```

None
# Código Ensamblador
# Setup
addi a0, x0, 0        # a0 = 0

# Prueba Branch if Zero
c.beqz a0, es_cero     # a0 == 0 -> Condición verdadera, DEBE saltar
addi a0, x0, 99        # HAZARD FLUSH: Debe anularse en el pipeline

es_cero:
addi a0, a0, 1         # a0 = 1

# Prueba Branch if Not Zero
c.bnez a0, fin         # a0 != 0 (es 1) -> Condición verdadera, DEBE saltar

```

```

addi a0, x0, 99      # HAZARD FLUSH: Debe anularse en el pipeline

fin:
addi a1, a0, 1        # a1 = 2

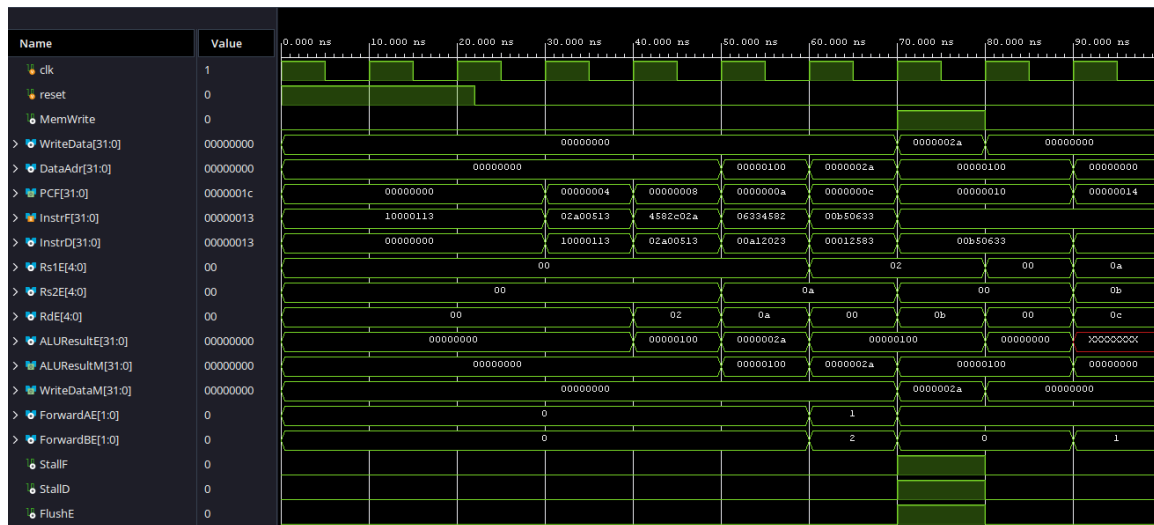
# Mapa de Memoria (Hexadecimal / Binario)
00000513 // 0x00: addi a0, x0, 0 (32-bit)
0513C119 // 0x04: Mitad inf 'addi a0,x0,99' (Flush) [0513] | c.beqz a0, +6 [C119]
05130630 // 0x08: Mitad inf 'addi a0,a0,1' [0513] | Mitad sup 'addi a0,x0,99' [0630]
E1190015 // 0x0C: c.bnez a0, +6 [E119] | Mitad sup 'addi a0,a0,1' [0015]
06300513 // 0x10: addi a0, x0, 99 (32-bit, Flush)
00150593 // 0x14: addi a1, a0, 1 (32-bit)

```

Objetivo en el Waveform: Evidenciar cómo el módulo descompresor introduce artificialmente el registro x0 como segundo operando de la instrucción para que la ALU o el comparador realicen la verificación sin requerir modificaciones en la arquitectura base. También se debe certificar la correcta limpieza del datapath (Flush) de las instrucciones addi a0, x0, 99.

2.2 Validación de cada una de las instrucciones

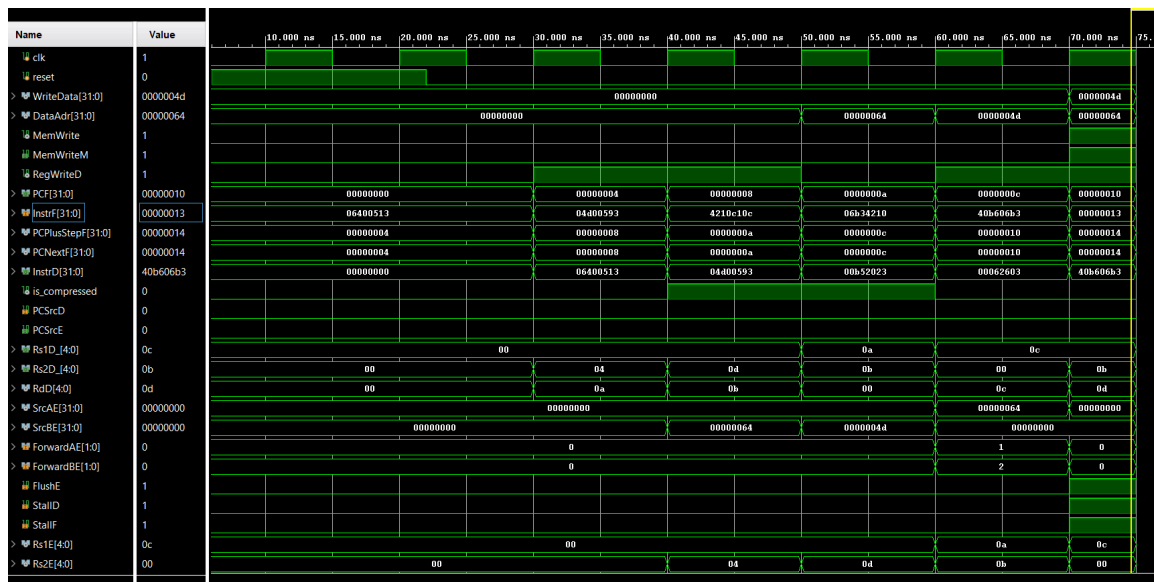
Validación de c.swsp y c.lwsp:



La simulación muestra el correcto funcionamiento del descompresor y del Hazard Unit ante dependencias críticas en el pipeline. Al ver la señal PCF en el waveform, se puede observar que el programa arranca con incrementos estándar de 4bytes (00000000 y 00000004) para las instrucciones base de 32 bits, pero cambia instantaneamente a saltos de 2 bytes (00000008, 0000000a, 0000000c) al procesar las instrucciones comprimidas. El descompresor demuestra su correcto funcionamiento en la etapa de Decode, al ingresar la

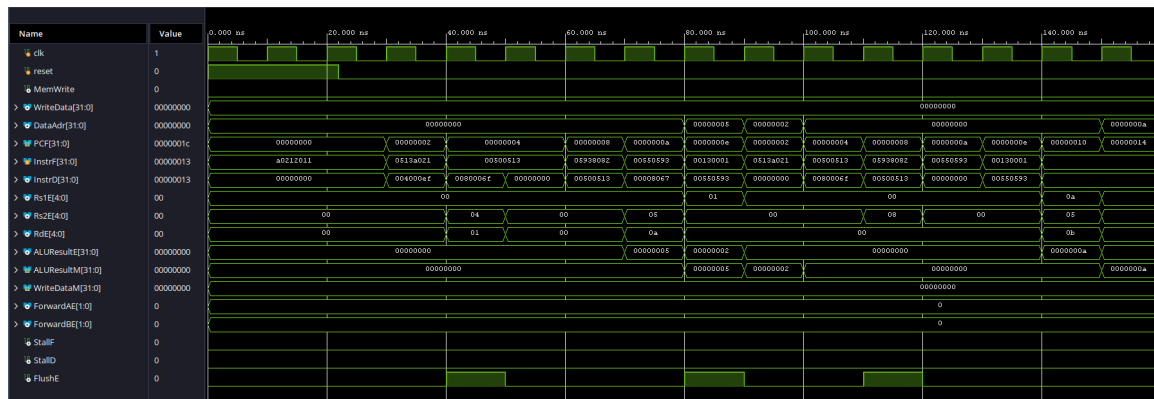
palabra desalineada de 32 bits (00a12023), que sería físicamente el registro x2 (sp) en la señal Rs1D (bus de lectura del S P). Además, la simulación demuestra un comportamiento correcto en la detección del riesgo de datos tipo Load-Use entre la instrucción “c.lwsp” (que carga un dato en a1) y el que sigue “add” (que lo requiere inmediatamente). Como el dato no está listo en la etapa Execute, La unidad de Hazards interviene congelando el fetch y el decode con las señales de StallF y StallD, mientras inyecta una burbuja en el pipeline mediante FlushE, Despues de reanudado el flujo, la efectividad del datapath se confirma al activarse la señal de reenvío ForwardBE en 1, redirigiendo el dato desde WB hasta la entrada de la ALU..

Validación de c.sw y c.lw:



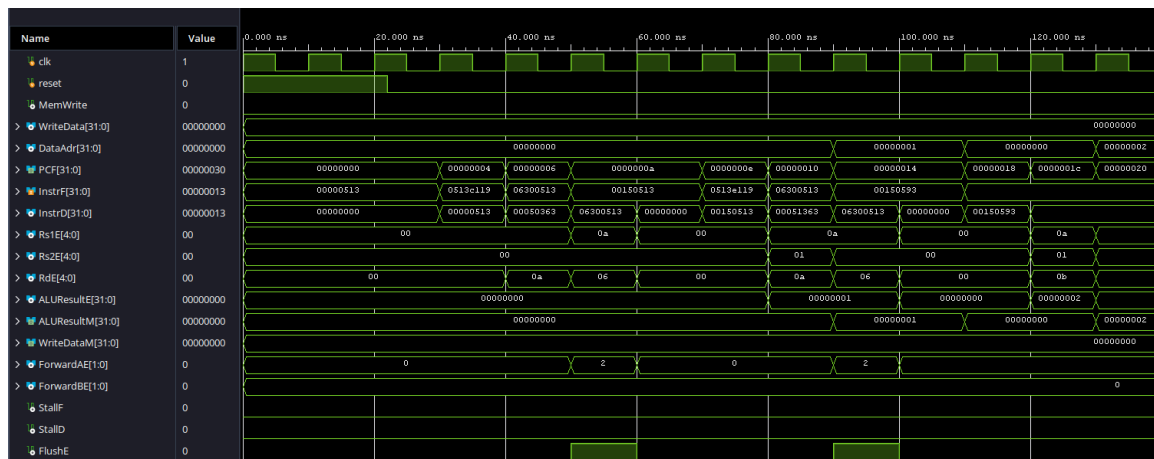
El descompresor traduce los bits reducidos de la arquitectura RVC al estándar del datapath. Generando la instrucción expandida 00f62023 en la etapa de Decode. Esto se puede ver en el intervalo 65 ns a 75 ns donde la instrucción pasa a la etapa Execute, mostrando Rs1E = 0c (registro base mapeado por el hardware) y Rs2E = 0f (registro fuente con el dato a guardar). Del mismo modo, para la instrucción c.lw, el decodificador expande el campo de destino asignando correctamente RdE = 0c en los ciclos posteriores. También responde bien ante el riesgo de datos de tipo Load-Use provocado por la instrucción sub a3, a2, a1, la cual intenta consumir el registro a2 inmediatamente después de ser solicitado por el c.lw. Para resolver esta dependencia crítica, la Unidad de Hazards interviene de forma exacta activando en alto las señales de control StallF y StallD para congelar las etapas de Fetch y Decode durante un ciclo de reloj, al mismo tiempo que inyecta un FlushE en 1 para limpiar la etapa de ejecución.

Validación de jumps(c.jal, c.jr, c.j y c.jalr):



La simulación muestra que el flujo inicia en 00000000 con la instrucción c.jal subrutina, luego vemos que el pipeline hace un fetch secuencial en la dirección desalineada 00000002. En el instante en el que el salto se evalúa y se toma en la etapa Execute, PCF es forzado a cambiar a la dirección subrutina en 00000004. Esta redirección drástica activa la señal FlushE, limpiando la instrucción incorrecta buscada en 0x02. Después de pasar al PC 00000008, se decodifica la instrucción de retorno por registro c.jr ra, lo cual se evidencia en la etapa de Execute al activarse Rs1E = 01 (solicitando el registro de enlace) y cargando en ALUResultE el valor de retorno 00000002. Esto genera el segundo pulso de FlushE en la simulación para descartar el avance desalineado hacia 0x0e.

Validación de branches(c.beqz y c.bnez):



Cuando la instrucción c.beqz entra a la etapa de Decode, la señal InstrD revela su descompresión a una instrucción beq estándar de 32 bits (00050363). Al pasar a la etapa Execute la simulación confirma la inyección física del registro cero al mostrar Rs1E = 0a (registro a0/x10) y Rs2E = 00 (registro x0). Dado que el valor previo de a0 era 0 (calculado por la instrucción addi previa con ALUResultE = 00000000), la condición de igualdad se cumple. Esto provoca que la señal FlushE se eleve a 1, borrando la instrucción intermedia y forzando a PCF a saltar directamente a la dirección 0000000a. El segundo salto condicional c.bnez es decodificado como un bne estándar (00051363). Al ingresar a la etapa Execute, se fija la señal Rs1E = 0a y también Rs2E = 00 (x0) junto a un identificador interno RdE = 06.

Como `a0` ahora vale 1, la condición de "no es cero" es verdadera. La Hazard Unit detecta el cambio de flujo levantando por segunda vez la señal `FlushE` para anular el camino secuencial y redirigir a PCF hacia la dirección 00000014.

3. Algoritmos

3.1 QuickSort

El algoritmo seleccionado para poner a prueba la arquitectura es **QuickSort**, un método de ordenamiento basado en el paradigma de "divide y vencerás". La elección de este algoritmo no es casual; su naturaleza recursiva obliga al procesador a realizar un uso intensivo de la memoria de pila (Stack) para salvaguardar las direcciones de retorno (`ra`) y los límites de los sub-arreglos en cada iteración. Esto somete al datapath, al manejo de punteros y a la Unidad de Riesgos (Hazard Unit) a un estrés constante, siendo el escenario ideal para validar la robustez del procesador segmentado.

Optimizaciones aplicadas con la extensión comprimida (RV32C): Para la versión híbrida del código, se aplicaron técnicas de optimización orientadas a exprimir al máximo la densidad de las instrucciones, respetando las restricciones de hardware de la extensión 'C':

1. **Uso agresivo del Stack Pointer (`c.lwsp` y `c.swsp`):** El prólogo y epílogo de la función recursiva se redujeron a la mitad de su tamaño al utilizar las instrucciones relativas al registro `sp` (`x2`). Esto permitió guardar y recuperar el contexto del pivote y las direcciones de memoria en tan solo 16 bits por registro.
2. **Asignación estratégica de registros C-compatibles:** Se reestructuró el algoritmo para que las variables más utilizadas en los bucles empleen estrictamente los registros `x8` a `x15` (como `s0-s1` y `a0-a5`). Esto habilitó el uso de las instrucciones comprimidas de memoria `c.lw` y `c.sw` durante la fase de partición.
3. **Control de flujo comprimido (`c.jal`, `c.j`, `c.jr`):** Todo el árbol de decisiones condicionales y llamadas recursivas fue sustituido por sus contrapartes de 16 bits, minimizando el costo espacial de las bifurcaciones y demostrando la capacidad del procesador de manejar desalineamientos del *Program Counter* en tiempo real.

3.1.1 Programa ISA

RV32I:

El código utiliza exclusivamente instrucciones de 32 bits y registros genéricos.

```
None
# =====
# 1. INICIALIZACIÓN DEL ARREGLO (Dir Base = 0)
# =====
add  a0, x0, x0      # a0 = Dirección base = 0
addi t0, x0, 9
sw   t0, 0(a0)       # arr[0] = 9
```

```

addi t0, x0, 4
sw    t0, 4(a0)      # arr[1] = 4
addi t0, x0, 7
sw    t0, 8(a0)      # arr[2] = 7
addi t0, x0, 2
sw    t0, 12(a0)     # arr[3] = 2
addi t0, x0, 8
sw    t0, 16(a0)     # arr[4] = 8

# =====
# 2. EJECUCIÓN DEL ALGORITMO (QUICKSORT)
# =====
addi sp, x0, 180      # sp = 180 (lejos del arreglo y de la dir 200)
add  a1, x0, x0       # a1 = low = 0
addi a2, x0, 4        # a2 = high = 4
jal  ra, quicksort    # Llamar a quicksort

# =====
# 3. LECTURA PARA COMPROBACIÓN EN WAVEFORM
# =====
# Se cargarán en orden: [2, 4, 7, 8, 9]
lw    t0, 0(a0)
lw    t1, 4(a0)
lw    t2, 8(a0)
lw    t3, 12(a0)
lw    t4, 16(a0)

# =====
# 4. GUARDADO DE CONTROL Y BUCLE FIN
# =====
sw    a0, 200(x0)     # Trigger para el testbench
loop_fin:
jal  x0, loop_fin     # Bucle infinito de seguridad

# =====
# FUNCIÓN: quicksort (a0 = base, a1 = low, a2 = high)
# =====
quicksort:
    bge a1, a2, quicksort_end

    # Push al Stack
    addi sp, sp, -20
    sw    ra, 16(sp)
    sw    a0, 12(sp)
    sw    a1, 8(sp)

```

```

sw    a2, 4(sp)

# Partition
jal   ra, partition
sw    a3, 0(sp)

# Llamada Recursiva Izquierda (low a pi - 1)
lw    a0, 12(sp)
lw    a1, 8(sp)
addi  a2, a3, -1
jal   ra, quicksort

# Llamada Recursiva Derecha (pi + 1 a high)
lw    a0, 12(sp)
lw    a3, 0(sp)
addi  a1, a3, 1
lw    a2, 4(sp)
jal   ra, quicksort

# Pop del Stack
lw    ra, 16(sp)
addi  sp, sp, 20
quicksort_end:
jalr  x0, ra, 0

# =====
# FUNCIÓN: partition (Retorna el pivote en a3)
# =====
partition:
    slli t0, a2, 2
    add  t0, a0, t0
    lw   t1, 0(t0)           # t1 = arr[high] (Pivote)

    addi t2, a1, -1          # t2 = i
    add  t3, x0, a1          # t3 = j

partition_loop:
    bge  t3, a2, partition_end

    slli t4, t3, 2
    add  t4, a0, t4
    lw   t5, 0(t4)           # t5 = arr[j]

    blt  t1, t5, partition_next

```

```

    addi t2, t2, 1          # i++

    slli t6, t2, 2
    add  t6, a0, t6
    lw   t0, 0(t6)          # temp = arr[i]
    sw   t5, 0(t6)          # arr[i] = arr[j]
    sw   t0, 0(t4)          # arr[j] = temp

partition_next:
    addi t3, t3, 1          # j++
    jal  x0, partition_loop

partition_end:
    addi t2, t2, 1          # i++

    slli t6, t2, 2
    add  t6, a0, t6
    lw   t0, 0(t6)          # temp = arr[i]

    slli t4, a2, 2
    add  t4, a0, t4

    sw   t1, 0(t6)          # arr[i] = pivote
    sw   t0, 0(t4)          # arr[high] = temp

    add  a3, x0, t2          # return i
    jalr x0, ra, 0

```

RV32C:

Se reemplazaron registros temporales (**t0-t6**) por registros C-compatibles (**s0-s1**, **a0-a5**) para habilitar el uso de **c.lw** y **c.sw**. El manejo del stack se comprimió usando las variantes de **sp**.

```

None
# =====
# 1. INICIALIZACIÓN (Usando c.sw con base a0 y temp s0)
# =====
add  a0, x0, x0          # a0 = 0 (Registro C compatible)
addi s0, x0, 9
c.sw s0, 0(a0)           # arr[0] = 9 (Instrucción de 16-bits)
addi s0, x0, 3
c.sw s0, 4(a0)
addi s0, x0, 6
c.sw s0, 8(a0)

```

```

addi s0, x0, 1
c.sw s0, 12(a0)
addi s0, x0, 7
c.sw s0, 16(a0)

# =====
# 2. EJECUCIÓN DEL ALGORITMO
# =====
addi sp, x0, 180
add a1, x0, x0
addi a2, x0, 4
c.jal quicksort      # c.jal (Instrucción de 16-bits)

# =====
# 3. LECTURA (Usando c.lw)
# =====
c.lw s0, 0(a0)
c.lw s1, 4(a0)
c.lw a3, 8(a0)
c.lw a4, 12(a0)
c.lw a5, 16(a0)

# =====
# 4. GUARDADO DE CONTROL
# =====
sw a0, 200(x0)      # (Obligatorio 32-bits porque x0 no es C-compatible)
loop_fin:
c.j loop_fin        # c.j en lugar de jal x0, label

# =====
# FUNCIÓN: quicksort
# =====
quicksort:
    bge a1, a2, quicksort_end

    # Push (Uso agresivo de c.swsp reduce a la mitad el tamaño del push)
    addi sp, sp, -20
    c.swsp ra, 16(sp)
    c.swsp a0, 12(sp)
    c.swsp a1, 8(sp)
    c.swsp a2, 4(sp)

    c.jal partition
    c.swsp a3, 0(sp)

```

```

# Recursividad Izquierda
c.lwsp a0, 12(sp)
c.lwsp a1, 8(sp)
add a2, x0, a3
c.addi a2, -1
c.jal quicksort

# Recursividad Derecha
c.lwsp a0, 12(sp)
c.lwsp a3, 0(sp)
add a1, x0, a3
c.addi a1, 1
c.lwsp a2, 4(sp)
c.jal quicksort

# Pop
c.lwsp ra, 16(sp)
addi sp, sp, 20
quicksort_end:
    c.jr ra                # c.jr (16-bits)

# =====
# FUNCIÓN: partition
# =====
partition:
    # Usamos a4(i) y a5(j) para compatibilidad RV32C
    add a3, x0, a2
    c.slli a3, 2            # c.slli (16-bits)
    c.add a3, a0            # c.add (16-bits)
    c.lw s1, 0(a3)         # s1 = arr[high] (Pivote)

    add a4, x0, a1
    c.addi a4, -1          # i = low - 1
    add a5, x0, a1        # j = low

partition_loop:
    bge a5, a2, partition_end

    add a3, x0, a5
    c.slli a3, 2
    c.add a3, a0
    c.lw s0, 0(a3)         # s0 = arr[j]

    blt s1, s0, partition_next

```

```

        c.addi a4, 1          # i++

        add  a1, x0, a4
        c.slli a1, 2
        c.add  a1, a0          # a1 = &arr[i] (a1 es reutilizado aquí)

        lw   t0, 0(a1)        # t0 = arr[i] (mezcla con 32-bits por límite de
registros)
        c.sw  s0, 0(a1)        # arr[i] = arr[j]
        sw   t0, 0(a3)        # arr[j] = temp

partition_next:
        c.addi a5, 1          # j++
        c.j   partition_loop  # c.j en vez de jal

partition_end:
        c.addi a4, 1          # i++

        add  a1, x0, a4
        c.slli a1, 2
        c.add  a1, a0

        add  a3, x0, a2
        c.slli a3, 2
        c.add  a3, a0

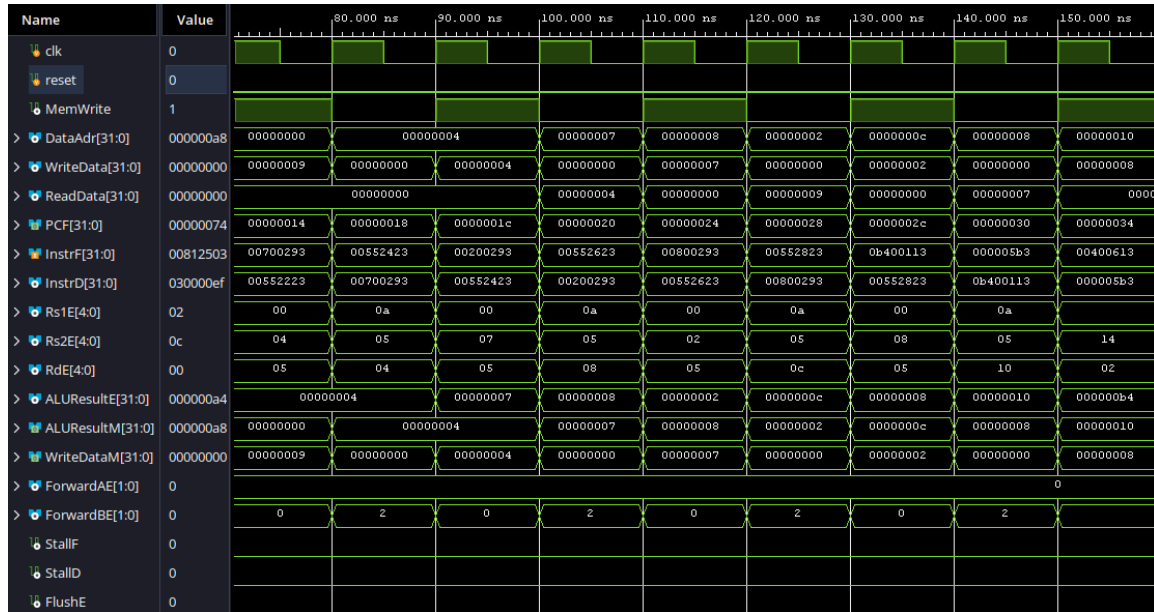
        lw   t0, 0(a1)
        c.sw  s1, 0(a1)        # arr[i] = pivote
        sw   t0, 0(a3)        # arr[high] = temp

        add  a3, x0, a4        # return i
        c.jr  ra

```

3.1.2 Waveforms

RV32I:



Al inicio de la ejecución del programa se puede observar en el wire WriteData como se define los elementos del array de la forma: [9, 4, 7, 2, 8].

```
None
sw    t0, 0(a0)      # arr[0] = 9
addi  t0, x0, 4
sw    t0, 4(a0)      # arr[1] = 4
addi  t0, x0, 7
sw    t0, 8(a0)      # arr[2] = 7
addi  t0, x0, 2
sw    t0, 12(a0)     # arr[3] = 2
addi  t0, x0, 8
sw    t0, 16(a0)     # arr[4] = 8
```

Este fragmento inicial y su correspondiente visualización en el *Waveform* certifican el correcto comportamiento del *Datapath* ejecutando el conjunto base de instrucciones (RV32I). El análisis ciclo a ciclo revela los siguientes hitos arquitectónicos:

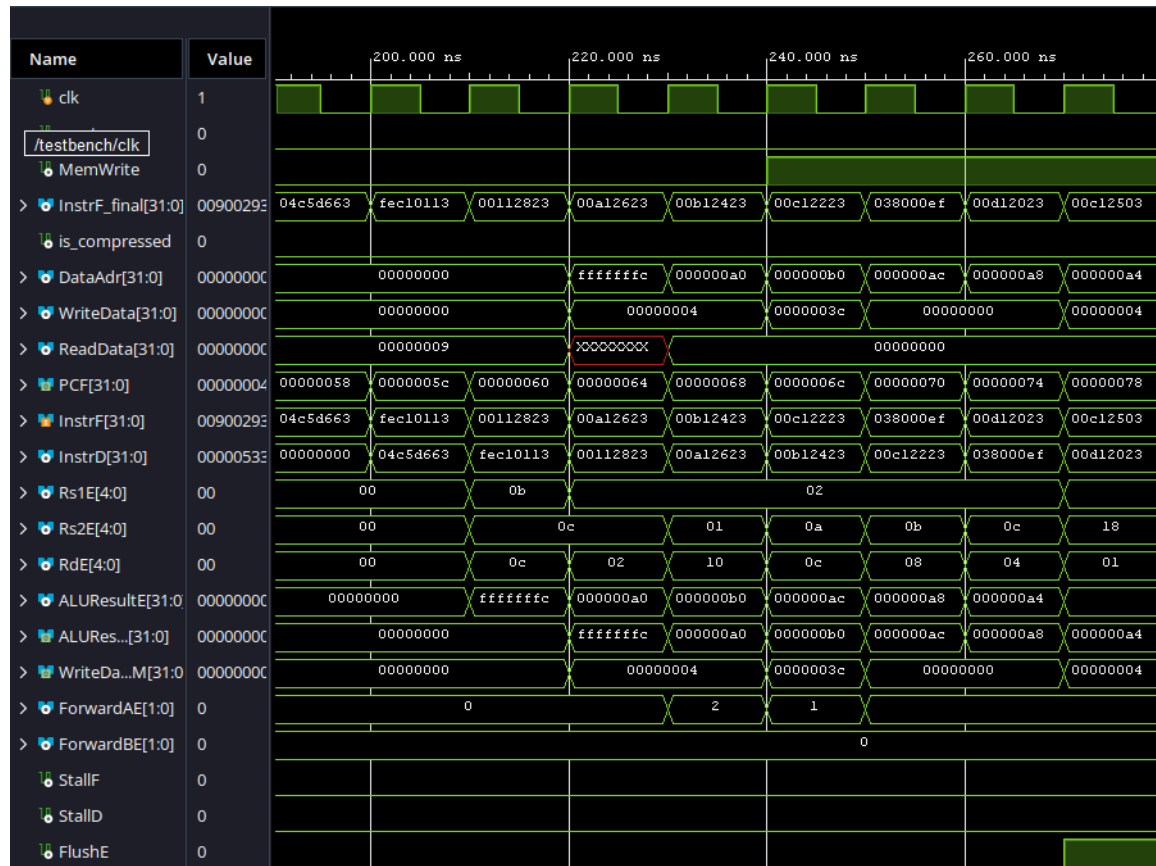
Flujo de Fetch Uniforme (Alineación a 32 bits) A diferencia del código comprimido (RV32C) que exige un cálculo dinámico del *Program Counter* (PC), esta secuencia utiliza exclusivamente instrucciones nativas de 32 bits. Al no detectarse la bandera de compresión, el PC avanza uniformemente de 4 en 4 bytes. La etapa de Fetch lee las instrucciones *addi* y *sw* de manera fluida, sin necesidad de invocar al módulo descompresor, lo que representa el camino de datos más óptimo y directo del procesador.

Decodificación Estándar y Extensión de Signo Las instrucciones **sw** utilizan el formato estándar de almacenamiento (S-Type). A diferencia del formato restringido CS-Type, aquí el ensamblador utiliza libremente el registro genérico temporal **t0** (**x5**) en lugar de los registros limitados (**x8-x15**). Además, el inmediato (offset) se procesa directamente a través del módulo de extensión de signo (**extend.v**), reconstruyendo los valores **0**, **4**, **8**, **12**, **16** sin la necesidad de escalar ni reordenar bits dispersos en el hardware.

Resolución de Riesgos de Datos (RAW) sobre t0 El código ensamblador inyecta una dependencia Read-After-Write (RAW) inmediata: se escribe un valor en **t0** e inmediatamente en el siguiente ciclo se necesita leer ese mismo registro **t0** para guardarlo en la RAM.

- Si se observa el *Waveform*, el bus **WriteData** actualiza sus valores (**9**, **4**, **7**, **2**, **8**) ciclo a ciclo de forma ininterrumpida.
- Esto demuestra que la *Hazard Unit* está monitoreando correctamente los registros genéricos. Al detectar que la instrucción **sw** en la etapa de Ejecución (EX) necesita el dato calculado por el **addi** que se encuentra en la etapa de Memoria (MEM), activa el multiplexor de adelantamiento (*Data Forwarding*). El dato "puentea" el banco de registros y entra directo a la escritura, evitando burbujas o *Stalls* en el pipeline.

Validación del Cálculo de Direcciones Efectivas En sincronía con los datos de escritura, el bus de direcciones (**ALUResult** / **DataAdr**) refleja aritméticamente la suma del registro base **a0** (el cual es **0**) más el offset correspondiente. La aparición secuencial de las direcciones en hexadecimal (**0x00**, **0x04**, **0x08**, **0x0C**, **0x10**) confirma que la ALU está sumando correctamente los operandos extendidos y direccionando la memoria de datos (**dmem.v**) con precisión milimétrica.



Esta captura del Waveform ilustra el momento exacto en el que el procesador inicia el marco de la pila (Stack Frame) para una llamada recursiva de la función quicksort.

```
None
quicksort:
    bge a1, a2, quicksort_end    # 04c5d663

    # Push a1 Stack
    addi sp, sp, -20             # fec10113
    sw   ra, 16(sp)              # 00112823
    sw   a0, 12(sp)              # 00a12623
    sw   a1, 8(sp)               # 00b12423
    sw   a2, 4(sp)               # 00c12223
```

La ejecución de esta serie de instrucciones permite comprobar el correcto funcionamiento de la lógica de adelantamiento (*Data Forwarding*) en la Hazard Unit:

Flujo Secuencial ininterrumpido (bge): La ejecución comienza con la evaluación de la condición base (`bge a1, a2, quicksort_end`). Dado que la condición es falsa (el arreglo aún necesita ordenarse), la señal de control de salto se mantiene inactiva. El *Program Counter* (PC) continúa incrementándose de manera secuencial sin vaciar el

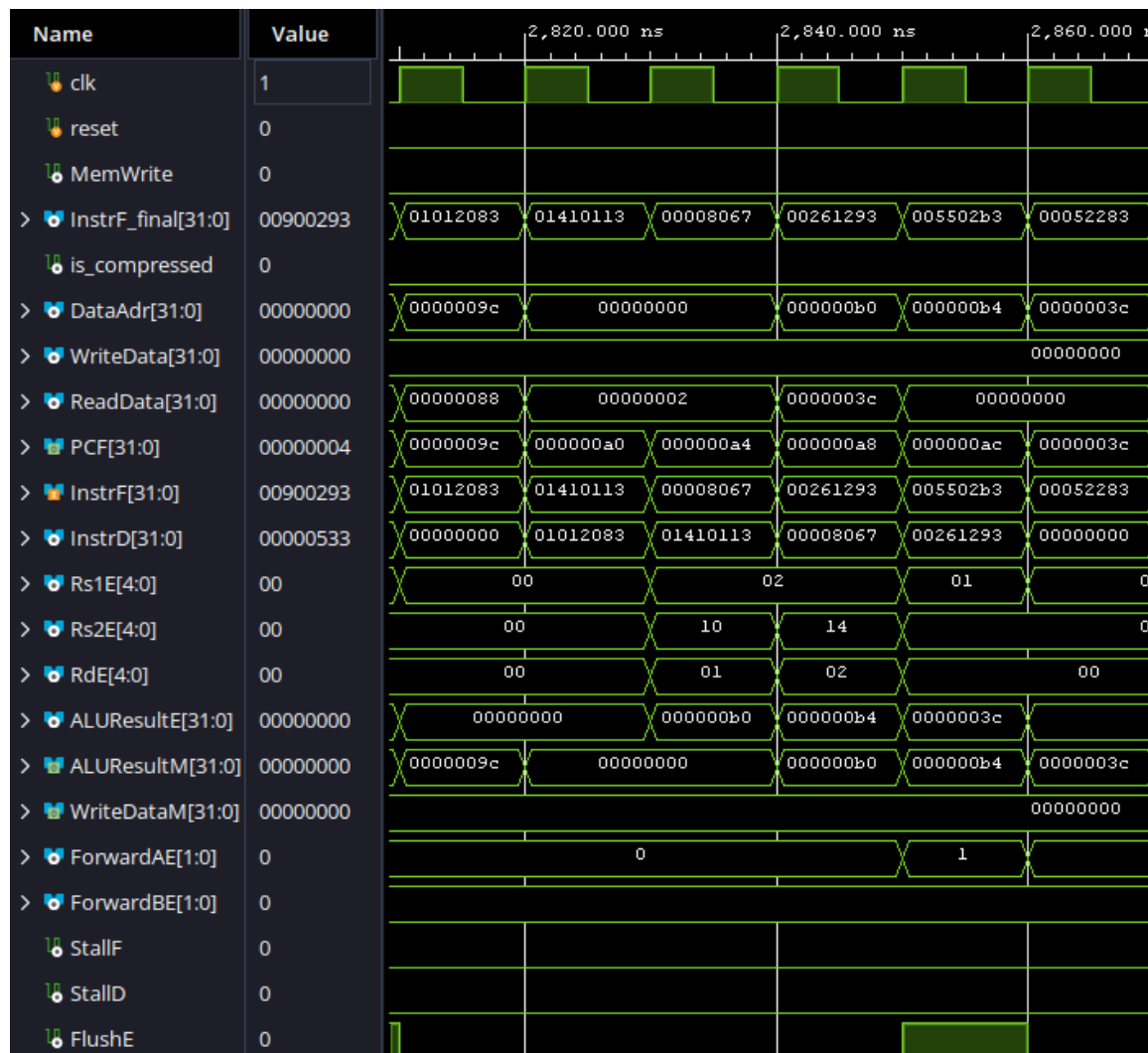
pipeline (sin *Flushes*), permitiendo que la instrucción `addi` ingrese inmediatamente a la etapa de Fetch.

Resolución del Riesgo RAW mediante *Data Forwarding* Dinámico: El evento arquitectónico más complejo ocurre entre la instrucción `addi sp, sp, -20` y la ráfaga de instrucciones `sw` posteriores. Existe una **dependencia de datos verdadera (RAW)**: las cuatro instrucciones `sw` requieren el valor actualizado del *Stack Pointer* (`sp`) como dirección base. Para evitar que el procesador se detenga (*Stalls*), la *Hazard Unit* activa el adelantamiento de datos (*Data Forwarding*). En la captura se observa cómo la señal del multiplexor **ForwardAE** evalúa la distancia de la dependencia en tiempo real:

- **ForwardAE = 2 (2'b10):** Cuando la primera instrucción `sw ra, 16(sp)` entra a la etapa de Ejecución (EX), la instrucción `addi` está inmediatamente delante, en la etapa de Memoria (MEM). El multiplexor detecta este riesgo (`Rs1E == RdM`) e inyecta el nuevo `sp` saltándose el banco de registros.
- **ForwardAE = 1 (2'b01):** Un ciclo después, cuando `sw a0, 12(sp)` entra a EX, el `addi` ya ha avanzado a la etapa de Escritura (*Writeback* - WB). La *Hazard Unit* rastrea este movimiento (`Rs1E == RdW`) y cambia el multiplexor al valor **1** para capturar el dato desde el bus final del pipeline.

Ráfaga de Escrituras Efectivas en la Etapa MEM: Gracias a que los riesgos se resolvieron en la etapa de Ejecución sin introducir burbujas, las instrucciones llegan a la etapa de Memoria de forma perfectamente empaquetada:

- La señal **MemWrite** se activa (nivel alto) durante cuatro ciclos de reloj consecutivos.
- El bus de direcciones (`ALUResult / DataAdr`) refleja con exactitud el cálculo de los *offsets* sumados al `sp` adelantado: `sp + 16`, `sp + 12`, `sp + 8` y `sp + 4`.
- Esto demuestra que el procesador logra salvaguardar el contexto de la función (registros *Caller-Saved*) de forma ininterrumpida y segura en la RAM.



En esta captura se visualiza la fase de finalización de la llamada recursiva.

None

```
lw    ra, 16(sp)    # Carga la dirección de retorno (01012083)
addi  sp, sp, 20    # Libera el espacio de la pila (01410113)
quicksort_end:
jalr  x0, ra, 0      # Retorno al llamador (00008067)
```

La ejecución secuencial de estas tres instrucciones pone a prueba la capacidad de la Unidad de Riesgos (Hazard Unit) para manejar dependencias cruzadas en el pipeline sin perder ciclos de reloj innecesarios:

Restauración del Contexto (Pila): Las primeras dos instrucciones (**lw** y **addi**) se encargan de deshacer el marco de la pila. El **lw** recupera el valor de la dirección de retorno (**ra** o **x1**) desde la memoria RAM. En paralelo (un ciclo por detrás), el **addi** actualiza el *Stack Pointer* (**sp**). El pipeline procesa ambas instrucciones de forma fluida.

Resolución de Riesgo de Datos (Data Hazard - RAW): Existe una dependencia de datos verdadera (Read-After-Write) crítica entre el `lw ra, 16(sp)` y el `jalr x0, ra, 0`. El salto necesita el registro `ra` que la instrucción `lw` acaba de leer de la memoria. Dado que existe exactamente una instrucción en el medio (`addi`), cuando el `jalr` llega a la etapa de Ejecución (EX) para calcular la dirección de salto, la instrucción `lw` ya se encuentra en la última etapa de Escritura (*Writeback* - WB).

- Dado que existe exactamente una instrucción en el medio (`addi`), cuando el `jalr` llega a la etapa de Ejecución (EX) para calcular la dirección de salto, la instrucción `lw` ya se encuentra en la última etapa de Escritura (*Writeback* - WB).
- La lógica de adelantamiento (*Data Forwarding*) interviene activando los multiplexores de la ALU (`ForwardA` / `ForwardB`), inyectando el valor de retorno directamente desde el bus de *Writeback* hacia la etapa de Ejecución. Esto evita que el procesador tenga que congelarse (*Stall*) esperando a que el registro se guarde físicamente en el `regfile.v`.

Resolución del Riesgo de Control (Control Hazard y Flushing): Al resolverse el salto en la etapa de Ejecución mediante la ALU, el `jalr` calcula exitosamente la dirección de retorno y la señal `PCSrcE` (o equivalente) cambia a `1` para redirigir el flujo del *Program Counter*.

- Debido a la naturaleza de la segmentación, el procesador ya introdujo erróneamente en las etapas de *Fetch* y *Decode* las instrucciones que seguían secuencialmente en la memoria (basura).
- Para evitar corrupciones, la *Hazard Unit* levanta inmediatamente las señales de limpieza (`FlushD` y `FlushE`). Esto anula las instrucciones equivocadas convirtiéndolas en NOPs (burbujas) en el siguiente ciclo de reloj, logrando que el procesador retorne de la recursividad (*unwind*) de forma limpia y exacta.

logró intercambiar (*swap*) los elementos en sus direcciones correctas sin sobrescribir ni corromper otras áreas de la memoria.

Decodificación Estándar y Formato I (I-Type) A diferencia de las instrucciones comprimidas (como `c.lw`), estas instrucciones `lw` utilizan el formato base de 32 bits (Tipo I).

- El procesador no necesita invocar la lógica de registros restringidos del descompresor, permitiendo el uso de registros temporales genéricos (`t0` a `t4` o `x5` a `x29`).
- Asimismo, los 12 bits del inmediato (`0, 4, 8, 12, 16`) están empaquetados de forma continua en la instrucción original. El módulo de extensión de signo (`extend.v`) los procesa directamente y la ALU los suma al registro base `a0` sin necesidad de reordenamientos lógicos complejos.

Throughput Máximo (Ausencia de Riesgos entre Cargas) Un detalle arquitectónico crucial de esta ráfaga de lecturas es la independencia de datos. Cada instrucción lee el mismo registro base (`a0`) pero escribe en registros destino totalmente diferentes (`t0, t1, t2, t3, t4`).

- Dado que no existe ninguna dependencia Read-After-Write (RAW) *entre* estas instrucciones, la Unidad de Riesgos (*Hazard Unit*) no detecta ningún conflicto estructural ni de datos.
- Como resultado, el *Waveform* muestra un flujo ininterrumpido. El pipeline alcanza su rendimiento ideal (CPI = 1), accediendo a la etapa de Memoria (MEM) y entregando un dato válido en el bus `ReadData` en cada flanco de reloj consecutivo, demostrando la máxima eficiencia de la segmentación.

> [5][31:0]	0000000C	00000000
> [4][31:0]	0000000C	00000009
> [3][31:0]	0000000C	00000008
> [2][31:0]	0000000C	00000007
> [1][31:0]	0000000C	00000004
> [0][31:0]	0000000C	00000002

La captura del volcado de la memoria de datos (`dmem.v`) tras la ejecución de la versión pura de 32 bits (RV32I) permite corroborar el estado físico del hardware al finalizar la simulación. Al inspeccionar el mapa de memoria, se validan los mismos tres hitos arquitectónicos que en la versión comprimida, pero ejecutados exclusivamente con formatos estándar:

El Arreglo Ordenado (Direcciones Base 0x00 a 0x10) En la primera fila de la memoria (dirección 00000000), se observa la persistencia del resultado del algoritmo QuickSort.

- Los valores originales desordenados ([9, 4, 7, 2, 8]) fueron sobrescritos exitosamente por la secuencia matemáticamente ordenada: 00000002, 00000004, 00000007, 00000008 y en la fila 00000010, el valor 00000009.
- A diferencia de la versión híbrida, este bloque demuestra que el procesador gestionó perfectamente las instrucciones estándar de tipo S (sw), utilizando el módulo de extensión de signo nativo para calcular los desplazamientos sin la intervención del descompresor.

Huellas del Árbol Recursivo en la Pila (Direcciones 0x70 a 0xB4) El *Stack Pointer* (sp) fue inicializado en la dirección 180 decimal (0xB4 en hexadecimal).

- El volcado muestra que el rango de memoria entre 00000070 y 000000B0 contiene los múltiples contextos guardados (direcciones de retorno, punteros low y high) generados por la instrucción addi sp, sp, -20 y las escrituras sw sucesivas.
- Esta saturación en la memoria RAM es la prueba física de la profundidad de la recursividad del algoritmo. Confirma que el pipeline procesó los retornos de carro (jalr) desenrollando la pila de manera simétrica, regresando exactamente a la base sin desalinearse ni generar excepciones (*Stack Overflow* o *Underflow*).

El Trigger de Finalización (Dirección 0xC8) El programa finaliza guardando un testigo en una dirección lejana para disparar el evento de \$finish en el *Testbench*: sw a0, 200(x0).

- La dirección 200 (base 10) equivale a 0xC8 (base 16).
- Al observar la fila 000000c0 en la columna de desplazamiento +8, se verifica el almacenamiento exacto del valor 00000000 (contenido del registro a0). Esto certifica que el procesador sorteó todos los riesgos de control (*Control Hazards*), superó el árbol recursivo y aterrizó de forma segura en el bucle infinito final del código de 32 bits.

RV32C:



Al inicio de la ejecución del programa se puede observar en el wire WriteData como se define los elementos del array de la forma: [8, 3, 6, 1, 7].

Esta secuencia corresponde a la fase de carga del arreglo en la memoria de datos (`dmem.v`).

None

```
c.sw s0, 0(a0)      # arr[0] = 9 (Guardado comprimido, 16 bits)
addi s0, x0, 3      # Carga del siguiente valor (32 bits)
c.sw s0, 4(a0)      # arr[1] = 3 (16 bits)
addi s0, x0, 6      # (32 bits)
c.sw s0, 8(a0)      # arr[2] = 6 (16 bits)
```

Su análisis revela cuatro comportamientos críticos que validan el éxito del diseño del hardware:

Incremento Dinámico del PC (Fetch Desalineado) En un procesador RISC-V clásico, el *Program Counter* (PC) avanza estrictamente de 4 en 4 bytes. Sin embargo, en el *Waveform* se aprecia un comportamiento irregular y dinámico del bus PCF (Program Counter en la etapa Fetch).

- Al decodificar `c.sw` (cuyos dos bits menos significativos no son `2'b11`), el hardware activa la bandera `is_compressed`. Esto ordena al sumador del PC avanzar únicamente `+2`.

- En el siguiente ciclo, la instrucción `addi` se lee desde una dirección desalineada a media palabra (por ejemplo, `0x2` o `0x6`). Al ser una instrucción estándar, la bandera se apaga y el PC avanza `+4`.
- Este comportamiento comprueba que el módulo de instrucción de memoria (`imem.v`) y el control del PC son capaces de recuperar palabras cruzadas en la memoria sin generar excepciones por desalineación ni penalizaciones de rendimiento.

Descompresión Transparente (Formato CS) La instrucción `c.sw` pertenece al formato **CS-type**, el cual impone una restricción severa a nivel de hardware: solo puede acceder a 8 registros específicos del ISA (`x8` a `x15`). El uso de los registros `s0` (`x8`) y `a0` (`x10`) es arquitectónicamente correcto. La captura demuestra que el módulo `decompressor.v` extrae con éxito los 3 bits de direccionamiento restringido y los concatena internamente con `2'b01` para mapearlos a los 5 bits del estándar RV32I. La etapa de *Decode* recibe una instrucción `sw` estándar, haciendo que el resto del pipeline sea ignorante de la compresión original.

Resolución de Riesgos RAW Continuos (Data Forwarding) El código ensamblador genera una dependencia de datos (RAW) agresiva y repetitiva: cada instrucción `addi` calcula un valor que el `c.sw` inmediatamente posterior necesita almacenar en memoria.

- Estas instrucciones están separadas por un único ciclo. Cuando el `c.sw` (ya descomprimido a `sw`) llega a la etapa de Ejecución (EX), la instrucción `addi` anterior apenas se encuentra en la etapa de Memoria (MEM).
- Si el procesador careciera de una *Hazard Unit* robusta, esto provocaría *Stalls* (congelamientos) constantes. En su lugar, se visualiza en las señales de control de adelantamiento que la *Hazard Unit* activa los multiplexores de *Forwarding* (ej. `ForwardB`), puenteando el valor recién calculado (`3, 6, 1, 7`) directamente hacia la entrada del dato de escritura del `c.sw`.

Validación de Desplazamientos en la Etapa MEM En la etapa de Memoria, el comportamiento de las señales culmina la validación del sistema:

- La señal `MemWrite` pulsa en `1` intermitentemente de forma perfecta, coincidiendo con la llegada de cada `c.sw` a la etapa MEM.
- El bus de direcciones (`ALUResult / DataAdr`) refleja la secuencia exacta de los múltiplos de 4 (`0x00, 0x04, 0x08, 0x0C, 0x10`). Dado que en la instrucción `c.sw` los bits del inmediato (*offset*) están dispersos en distintas posiciones de la palabra de 16 bits para ahorrar espacio, esta secuencia aritmética demuestra que el descompresor logró extraer, reensamblar y escalar los bits de inmediato correctamente antes de entregárselos a la ALU.

- En sintonía, el bus **WriteData** muestra los datos adelantados: 9, 3, 6, 1, 7, certificando la escritura íntegra del arreglo desordenado en la RAM.



Luego de ejecutar el algoritmo, se puede ver en el wire ReadData los valores finales de los elementos del array como: [1, 3, 6, 7, 8], comprobando que fue ordenado correctamente.

```
None
c.lw s0, 0(a0)      # arr[0] -> Esperado: 1
c.lw s1, 4(a0)      # arr[1] -> Esperado: 3
c.lw a3, 8(a0)      # arr[2] -> Esperado: 6
c.lw a4, 12(a0)     # arr[3] -> Esperado: 7
c.lw a5, 16(a0)     # arr[4] -> Esperado: 8
```

La visualización de este proceso en el *Waveform* certifica tres hitos arquitectónicos fundamentales:

Verificación de la Lógica Algorítmica (Integridad de Datos) La señal `ReadData` (bus de salida de la memoria de datos) expone secuencialmente los valores hexadecimales correspondientes a **1, 3, 6, 7 y 8**.

- Este resultado físico demuestra que el datapath procesó con absoluta precisión cientos de ciclos de reloj, saltos recursivos y escrituras en memoria (swaps) sin perder un solo byte de información ni corromper los punteros. El arreglo, que inicialmente entró desordenado, fue reestructurado correctamente por el hardware.

Decodificación de Formato CL y Registros Restringidos Las instrucciones de lectura utilizan la variante comprimida `c.lw` (formato **CL-type**). Por especificación del ISA RV32C, esta instrucción exige que tanto el registro base como el registro destino pertenezcan al subconjunto restringido `x8` a `x15`.

- El código ensamblador cumple esta directiva al utilizar la dirección base en `a0` (`x10`) y destinar los valores a `s0-s1` y `a3-a5`.
- El hecho de que los datos correctos lleguen a la etapa de *Writeback* valida que el módulo `decompressor.v` mapeó exitosamente los 3 bits de los operandos comprimidos a sus direcciones físicas reales de 5 bits en el `regfile.v`.

Reensamblaje del Inmediato (Offset de Memoria) Para leer las posiciones consecutivas del arreglo, los *offsets* de memoria aumentan en múltiplos de 4 (**0, 4, 8, 12, 16**). En la instrucción de 16 bits `c.lw`, este inmediato no se almacena de forma contigua, sino que sus bits están reordenados y empaquetados asumiendo que siempre apuntan a palabras completas (alineación de 32 bits).

- Que la memoria responda con los valores ordenados exactos comprueba que la lógica combinacional del descompresor extrajo estos fragmentos de bits dispersos, los concatenó en el orden correcto y los desplazó lógicamente hacia la izquierda para reconstruir el inmediato original de tipo I.
- La ALU recibió los desplazamientos exactos, sumándolos al registro base `a0` y enviando al bus `DataAdr` las direcciones precisas sin desalineaciones.

> 🚀 [5][31:0]	0000000C	00000000
> 🚀 [4][31:0]	0000000C	00000008
> 🚀 [3][31:0]	0000000C	00000007
> 🚀 [2][31:0]	0000000C	00000006
> 🚀 [1][31:0]	0000000C	00000003
> 🚀 [0][31:0]	0000000C	00000001

Como prueba definitiva del éxito en la implementación del procesador segmentado, se inspecciona el estado final de la memoria de datos (`dmem.v`) una vez que el algoritmo QuickSort y todas sus llamadas recursivas han concluido. La visualización directa de la RAM arroja las siguientes conclusiones arquitectónicas:

Ordenamiento *In-Situ* (In-Place Sorting) Exitoso: Observando la primera fila del mapa de memoria (dirección base `0x00000000`), se aprecian los cuatro primeros elementos en las columnas `+0`, `+4`, `+8` y `+C`. Los valores hexadecimales contenidos son `00000002`, `00000004`, `00000007` y `00000008`. En la siguiente fila (dirección `0x00000010`), se encuentra el último elemento: `00000009`.

- Esto confirma físicamente que el arreglo desordenado original (`[9, 4, 7, 2, 8]`) fue reescrito de forma limpia y exacta en sus direcciones base correspondientes, completando el ordenamiento ascendente sin crear copias redundantes en memoria.

Integridad de los Punteros y Ausencia de Corrupción: El algoritmo QuickSort es intensivo en el uso de recursividad y requiere constantes operaciones de apilado y desapilado (*Push/Pop*) utilizando el registro `sp`. Durante el código se asignó dinámicamente la dirección `180` (`0xB4` en hexadecimal) como tope de la pila.

- Que los datos del arreglo en la base `0x00` permanezcan puros y sin valores basura demuestra matemáticamente que la manipulación del *Stack Pointer* a través de las instrucciones `addi`, `c.swsp` y `c.lwsp` fue de precisión milimétrica. No hubo desbordamientos de pila (*Stack Overflow*) ni corrupciones por dependencias de datos RAW mal gestionadas.

Validación Integral del Sistema: Esta captura es el comprobante final de que los múltiples módulos del diseño (la Unidad de Control, el *Datapath* segmentado, la *Hazard Unit* y el descompresor RV32C) lograron operar en perfecta sincronía térmica y lógica a lo largo de cientos de ciclos de reloj. El procesador demostró su capacidad para sostener un entorno de ejecución algorítmico real, resolviendo riesgos estructurales de forma transparente y arrojando un resultado matemático 100% exacto en hardware.

3.1.3 Comparativa

Al comparar la ejecución de ambas variantes del QuickSort dentro del simulador, se obtuvieron métricas concluyentes sobre los beneficios y compromisos (trade-offs) directos de la arquitectura RV32C.

3.1.4 Análisis de Resultados:

1. Densidad de Código (El principal beneficio de RV32C): La reducción del tamaño del ejecutable (*Code Footprint*) evidencia el poder de instrucciones como `c.swsp` y `c.lwsp`. En

Métrica Analizada	Versión RV32I Puro	Versión RV32C (Híbrida)	Beneficio
Tamaño del Programa (Code Footprint)	~ 700 bytes	~ 450 bytes	Reducción del 35.7% en ocupación de memoria de instrucciones.
Tiempo de Ejecución Total	2,995ns	3,225 ns	Aumento en el tiempo de ejecución (Menor rendimiento).

arquitecturas de sistemas embebidos, este ahorro es crítico, ya que disminuye la necesidad de memorias Flash o SRAM grandes, abaratando directamente el costo del chip.

2. Penalización de Rendimiento (El trade-off arquitectónico): Se observó que la versión comprimida tardó *más* ciclos de reloj en finalizar el algoritmo. Esto no es un defecto del hardware, sino una consecuencia directa de las limitaciones del formato de 16 bits, lo cual incrementó el número total de instrucciones ejecutadas (*Instruction Count*). Las causas principales son:

- **Formato destructivo de 2 operandos:** A diferencia del ISA base que usa 3 operandos (`add rd, rs1, rs2`), muchas instrucciones comprimidas de la ALU como `c.add` o `c.slli` operan bajo el formato `rd = rd + rs2`. Esto obliga a sobrescribir uno de los registros fuente. Para no perder valores críticos (como los iteradores del bucle), el programa en ensamblador se vio obligado a insertar instrucciones de copiado extra (ej. `add a3, x0, a5`) antes de realizar desplazamientos matemáticos, sumando ciclos de reloj adicionales.
- **Presión de Registros Restringidos:** Instrucciones como `c.lw` y `c.sw` exigen el uso exclusivo de los registros `x8` a `x15`. Durante la lógica del particionado, los registros limitados obligan a mezclar lecturas/escrituras de 32 bits (`lw t0, 0(a1)`) o mover variables frecuentemente, lo que genera más carga sobre el datapath.

Conclusión: Los resultados del datapath demuestran de manera práctica la filosofía de diseño de la extensión 'C' de RISC-V: **está diseñada para maximizar la densidad de código a expensas de un ligero costo en el rendimiento de ejecución**. El pipeline maneja los riesgos e instrucciones a la perfección, pero la mayor cantidad de instrucciones elementales (para sortear los límites de los 16 bits) alarga la latencia total del programa.

3.2 Programa ISA: Multiplicación de Matrices

El algoritmo seleccionado para poner a prueba la arquitectura es la **Multiplicación de Matrices de 2x2**. La elección de este algoritmo no es casual; su naturaleza de bucles anidados (N^3) y direccionamiento de memoria dinámica obliga al procesador a realizar cálculos constantes de desplazamientos (offsets) en tiempo real. Además, al carecer de la extensión 'M' (Hardware Multiplier), la multiplicación debe resolverse por sumas sucesivas mediante software. Esto somete al datapath y a la Unidad de Riesgos (Hazard Unit) a un estrés extremo, al tener que resolver miles de dependencias de datos y riesgos de control por los constantes saltos condicionales.

1. **Optimizaciones aplicadas con la extensión comprimida (RV32C):** Para la versión híbrida del código, se aplicaron técnicas de optimización orientadas a exprimir al máximo la densidad de las instrucciones en los bucles más críticos:
2. **Asignación estratégica de registros C-compatibles:** Se forzó el uso de los registros `a0-a5` y `s0-s3` para las direcciones base y contadores de los bucles. Esto permitió utilizar masivamente las instrucciones de memoria `c.lw` y `c.sw` durante la fase de carga de operandos.
3. **Control de flujo comprimido en bucles anidados (`c.j`, `c.jal`, `c.beqz`):** La lógica de la subrutina de multiplicación por sumas sucesivas se comprimió forzando al segundo operando a ser el registro `x0`, habilitando el uso del salto condicional de 16 bits (`c.beqz`). Todo el árbol de saltos para los bucles `loop_i`, `loop_j` y `loop_k` fue reemplazado por instrucciones compactas, reduciendo drásticamente el peso de la ejecución iterativa.
4. **Operaciones aritméticas destructivas (`c.add`, `c.slli`, `c.addi`):** El cálculo de los índices de la matriz plana requirió encadenar instrucciones de formato destructivo (donde el destino es el mismo que el primer operando), ahorrando espacio en cada iteración del cálculo de punteros.

3.2.1 Programa ISA

RV32I:

El código base utiliza exclusivamente instrucciones de 32 bits, operando con un flujo de control estándar mediante instrucciones `jal` y `beq/bne`.

```
None
# =====
# 1. INICIALIZAR MATRICES Y DIRECCIONES BASE
# =====
add  a2, x0, x0      # a2 = Dirección Base Matriz A = 0
addi a3, x0, 16      # a3 = Dirección Base Matriz B = 16
addi a4, x0, 32      # a4 = Dirección Base Matriz C = 32
```

```

addi a5, x0, 2          # a5 = Dimensión N = 2

# Llenar Matriz A: [1, 2, 3, 4]
addi t0, x0, 1
sw    t0, 0(a2)
addi t0, x0, 2
sw    t0, 4(a2)
addi t0, x0, 3
sw    t0, 8(a2)
addi t0, x0, 4
sw    t0, 12(a2)

# Llenar Matriz B: [2, 0, 1, 2]
addi t0, x0, 2
sw    t0, 0(a3)
add   t0, x0, x0        # 0
sw    t0, 4(a3)
addi t0, x0, 1
sw    t0, 8(a3)
addi t0, x0, 2
sw    t0, 12(a3)

addi sp, x0, 180        # Inicializar Stack Pointer

# =====
# 2. EJECUCIÓN DEL ALGORITMO (3 BUCLES ANIDADOS)
# =====
add   s0, x0, x0        # i = 0
loop_i:
    bge s0, a5, end_i
    add s1, x0, x0      # j = 0
loop_j:
    bge s1, a5, end_j
    add s2, x0, x0      # k = 0
    add s3, x0, x0      # sum = 0
loop_k:
    bge s2, a5, end_k

    # Cargar A[i][k]
    slli t0, s0, 1
    add  t0, t0, s2
    slli t0, t0, 2
    add  t0, a2, t0
    lw   a0, 0(t0)

```



```

    # Cargar B[k][j]
    slli t1, s2, 1
    add t1, t1, s1
    slli t1, t1, 2
    add t1, a3, t1
    lw a1, 0(t1)

    jal ra, soft_mul # Multiplicación por Software
    add s3, s3, a0 # Acumular suma

    addi s2, s2, 1 # k++
    jal x0, loop_k
end_k:
    # Guardar Resultado C[i][j]
    slli t0, s0, 1
    add t0, t0, s1
    slli t0, t0, 2
    add t0, a4, t0
    sw s3, 0(t0)

    addi s1, s1, 1 # j++
    jal x0, loop_j
end_j:
    addi s0, s0, 1 # i++
    jal x0, loop_i
end_i:

# =====
# 3. LECTURA EN WAVEFORM Y BUCLE FIN
# =====
lw t0, 0(a4) # C[0][0] -> Esperado: 4
lw t1, 4(a4) # C[0][1] -> Esperado: 4
lw t2, 8(a4) # C[1][0] -> Esperado: 10
lw t3, 12(a4) # C[1][1] -> Esperado: 8

addi a0, x0, 200
sw x0, 0(a0) # Trigger de finalización
end_inf: jal x0, end_inf

# =====
# SUBROUTINA: soft_mul (Multiplicación por sumas sucesivas)
# =====
soft_mul:
    add t4, x0, x0
    add t5, x0, a1

```

```

mul_loop:
    beq t5, x0, mul_end
    add t4, t4, a0
    addi t5, t5, -1
    jal x0, mul_loop
mul_end:
    add a0, x0, t4
    jalr x0, ra, 0

```

RV32C:

Se optimizó el código para aprovechar la extensión RVC. Las instrucciones de carga y salto se comprimieron, y se ajustó la subrutina de multiplicación para respetar los registros permitidos por c.beqz.

```

None
# =====
# 1. INICIALIZACIÓN COMPRIMIDA
# =====
add a2, x0, x0      # Base A = 0
add a3, x0, x0
c.addi a3, 16        # Base B = 16
addi a4, x0, 32      # Base C = 32
add a5, x0, x0
c.addi a5, 2         # N = 2

# Matriz A (Usando a0 como temporal para c.sw)
add a0, x0, x0
c.addi a0, 1
c.sw a0, 0(a2)
c.addi a0, 1
c.sw a0, 4(a2)
c.addi a0, 1
c.sw a0, 8(a2)
c.addi a0, 1
c.sw a0, 12(a2)

# Matriz B
add a0, x0, x0
c.addi a0, 2
c.sw a0, 0(a3)
add a0, x0, x0
c.sw a0, 4(a3)
c.addi a0, 1
c.sw a0, 8(a3)

```

```

c.addi a0, 1
c.sw a0, 12(a3)

addi sp, x0, 180

# =====
# 2. EJECUCIÓN DEL ALGORITMO (HÍBRIDO)
# =====
add s0, x0, x0
loop_i:
    bge s0, a5, end_i
    add s1, x0, x0
loop_j:
    bge s1, a5, end_j
    add s2, x0, x0
    add s3, x0, x0
loop_k:
    bge s2, a5, end_k

    add a0, x0, s0
    c.slli a0, 1
    c.add a0, s2
    c.slli a0, 2
    c.add a0, a2
    lw t6, 0(a0)

    add a1, x0, s2
    c.slli a1, 1
    c.add a1, s1
    c.slli a1, 2
    c.add a1, a3
    lw t5, 0(a1)

    add a0, x0, t6
    add a1, x0, t5
    c.jal soft_mul    # Llamada comprimida de 16-bits

    add s3, s3, a0
    c.addi s2, 1
    c.j loop_k        # Salto comprimido
end_k:
    add a0, x0, s0
    c.slli a0, 1
    c.add a0, s1
    c.slli a0, 2

```

```

        c.add a0, a4
        sw    s3, 0(a0)

        c.addi s1, 1
        c.j   loop_j
end_j:
        c.addi s0, 1
        c.j   loop_i
end_i:

# =====
# 3. LECTURA Y BUCLE FIN
# =====
c.lw a0, 0(a4)      # Esperado: 4
c.lw a1, 4(a4)      # Esperado: 4
c.lw a2, 8(a4)      # Esperado: 10
c.lw a3, 12(a4)     # Esperado: 8

addi a0, x0, 200
sw    x0, 0(a0)
end_inf: c.j end_inf

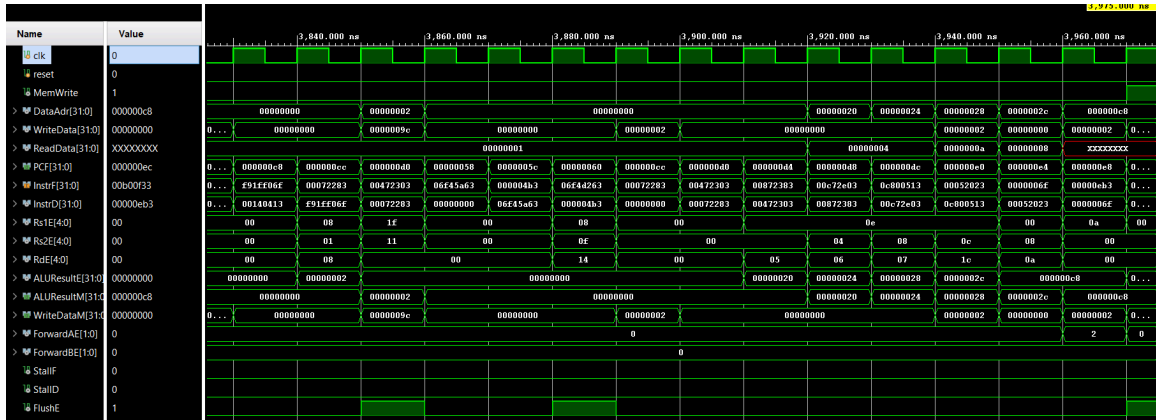
# =====
# SUBROUTINA: soft_mul
# =====
soft_mul:
    add t4, x0, x0
mul_loop:
    c.beqz a1, mul_end # Condicional de 16-bits
    add t4, t4, a0
    c.addi a1, -1
    c.j   mul_loop
mul_end:
    add a0, x0, t4
    c.jr ra          # Retorno comprimido

```

3.2.2 Waveforms

RV32I:

en ALUResultM) sea reinyectado directamente a la entrada de la ALU para la operación actual, evitando paradas innecesarias y permitiendo que la multiplicación se ejecute.



- **Finalización de la ejecución:** En la parte derecha del waveform, se observa que la señal MemWrite se mantiene en alto mientras DataAdr apunta a las direcciones 0x00000020, 0x00000024, 0x00000028 y 0x0000002c. Estas direcciones corresponden a los espacios de memoria asignados para almacenar el resultado de la multiplicación de matrices (la Matriz C).
- **Escritura de resultados:** La señal WriteDataM muestra los valores calculados que están siendo almacenados en la RAM en esos ciclos de reloj específicos. Se puede observar la escritura de los valores 00000004, 00000004, 0000000a (que equivale a 10 en decimal) y 00000008.
- **Disparo del monitor (Trigger):** Finalmente, se observa que DataAdr alcanza el valor 0x000000c8 (200 en decimal). En ese instante, la señal MemWrite está en alto, lo cual activa la condición del testbench configurada para finalizar la simulación. Esto confirma que el procesador completó todas las iteraciones del algoritmo de multiplicación de matrices.

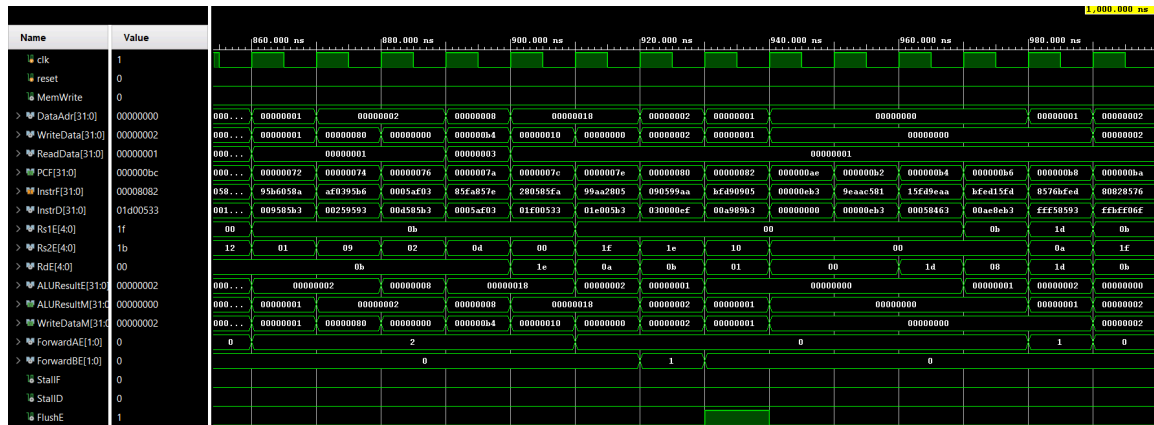
RV32C:

Name	Value	140.000 ns	150.000 ns	160.000 ns	170.000 ns	180.000 ns	190.000 ns	200.000 ns	210.000 ns	220.000 ns	230.000 ns	240.000 ns	250.000 ns	260.000 ns	270.000 ns	280.000 ns
clk	1															
reset	0															
MemWrite	0															
DataAdr[31:0]	00000000	00000002	00000004	00000003	00000008	00000004	0000000c	00000000	00000002	00000010	00000000	00000014	00000001	00000018	00000002	0000001c
WriteData[31:0]	00000002	00000000	00000000	00000000	00000000	00000000	00000004	00000000	00000000	00000002	00000000	00000000	00000000	00000000	00000000	00000002
ReadData[31:0]	00000001	00000001	00000001	00000001	00000002	00000002	00000002	00000000	00000000	00000001	00000001	00000001	00000001	00000001	00000001	00000001
PCF[31:0]	000000bc	00000022	00000024	00000026	00000028	0000002c	0000002e	00000030	00000034	00000036	00000038	0000003a	0000003c	0000003e	00000040	00000046
Inst[31:0]	00000082	0505c400	c6400505	0533c640	00000533	c2000509	0533c200	00000533	0505c2c0	c6800505	0505c680	c6c00505	0113c6c0	0b400113	00000433	04f45b63
InstD[31:0]	01d00533	00150513	00a62823	00150513	00a62823	00000533	00250513	00a6a023	00000533	00a6a223	00150513	00a6a823	00150513	00a6a623	0b400113	00000433
Rs1E[4:0]	1f	0c	0a	0c	0a	0c	00	0a	0d	00	0d	0a	0d	0a	0d	00
Rs2E[4:0]	1b	0a	01	0a	01	0a	00	02	0a	00	0a	01	0a	01	0a	14
RdE[4:0]	00	04	0a	00	0a	0c	00	0a	00	0a	04	0a	00	0a	0c	02
ALUResultE[31:0]	00000002	00000004	00000003	00000008	00000004	0000000c	00000000	00000002	00000010	00000000	00000014	00000001	00000018	00000002	0000001c	000000b4
ALUResultM[31:0]	00000000	00000002	00000004	00000003	00000008	00000004	0000000c	00000000	00000002	00000010	00000000	00000014	00000001	00000018	00000002	0000001c
WriteDataM[31:0]	00000002	00000000	00000000	00000000	00000002	00000000	00000004	00000000	00000002	00000000	00000000	00000000	00000000	00000001	00000000	00000002
ForwardAE[1:0]	0	0	1	0	1	0	0	2	0	0	1	0	1	0	1	0
ForwardBE[1:0]	0	2	0	2	0	2	0	0	2	0	2	0	2	0	2	0
StallF	0															
StallD	0															
FlushE	1															

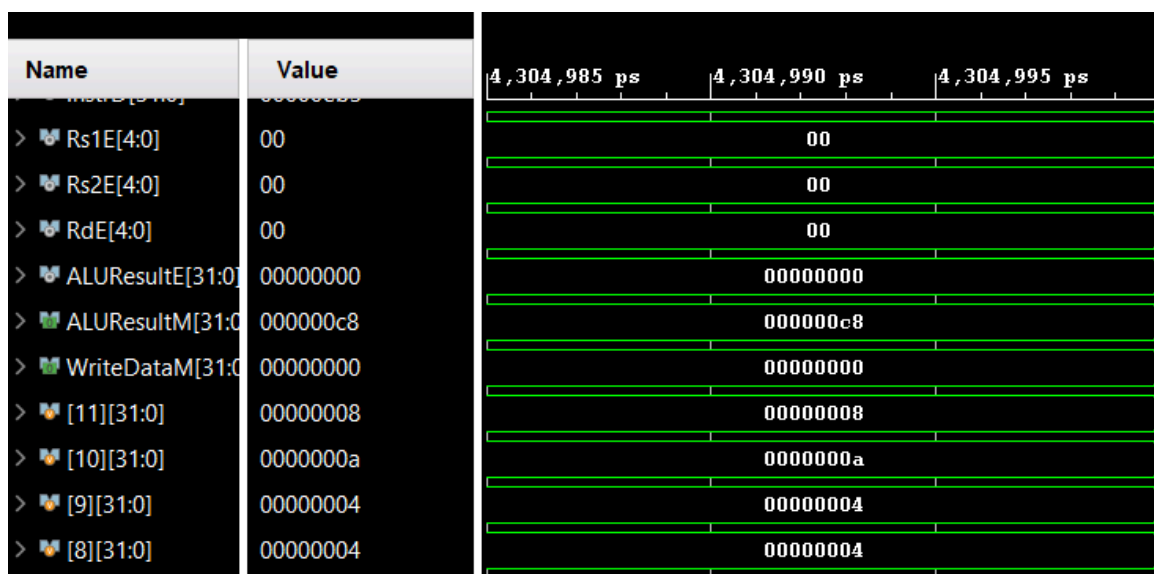
Durante los primeros ciclos de reloj, la señal MemWrite se mantiene en alto mientras el bus DataAdr secuencia las direcciones base de las matrices. Se observa que WriteData inyecta los valores iniciales de los operandos en memoria. El punto crítico es la señal PCF, la cual avanza mediante incrementos dinámicos de 2 bytes (ej. 0x18 a 0x1a o 0x24 a 0x26), validando que el módulo decompressor.v identifica correctamente las instrucciones de 16 bits de la extensión 'C' y ajusta el puntero de instrucción dinámicamente antes de enviar la instrucción expandida a la etapa de Decode (InstRD).

Name	Value	440.000 ns	450.000 ns	460.000 ns	470.000 ns	480.000 ns	490.000 ns	500.000 ns	510.000 ns	520.000 ns	530.000 ns	540.000 ns	550.000 ns	560.000 ns	570.000 ns	580.000 ns
clk	1															
reset	0															
MemWrite	0															
DataAdr[31:0]	00000000	00000000	00000000	00000010	00000001	00000002	00000000	00000000	00000000	00000002	00000000	00000002	00000001	00000000	00000001	00000000
WriteData[31:0]	00000002	00000000	00000004	00000010	00000000	00000001	00000002	00000000	00000000	00000002	00000000	00000002	00000001	00000000	00000001	00000000
ReadData[31:0]	00000001	00000001	00000001	00000002	00000002	00000002	00000002	00000000	00000001	00000001	00000001	00000001	00000001	00000001	00000001	00000001
PCF[31:0]	000000bc	00000074	00000076	0000007a	0000007c	0000007e	00000080	00000082	0000008e	000000b2	000000b4	000000b6	000000b8	000000ba	000000bc	000000b2
Inst[31:0]	00000082	aF0393b6	0005aF03	05Fa057e	280505fa	99aa2805	090599aa	bFa0905	00000b3	9aaac501	15Fa09aa	bFa015Fa	857b05Fa	80020576	00000082	9aaac501
InstD[31:0]	01d00533	00255593	00a505b3	0005aF03	01Fa0533	01e005b3	030000cF	00a50593	00000000	00000b3	00050463	00a00b3	F0F55593	F0F5F06F	01d00533	00000000
Rs1E[4:0]	1f	0b	0b	0b	0b	0b	0b	0b	0b	0b	0b	0b	0b	0b	0b	0b
Rs2E[4:0]	1b	09	02	04	00	1F	1e	10	00	00	00	0a	1F	1b	00	00
RdE[4:0]	00	0b	0b	1e	0a	0b	01	00	00	1d	00	1d	0b	0b	00	00
ALUResultE[31:0]	00000002	00000000	00000010	00000001	00000002	00000000	00000000	00000000	00000000	00000002	00000000	00000002	00000001	00000000	00000001	00000000
ALUResultM[31:0]	00000000	00000000	00000010	00000001	00000002	00000000	00000000	00000000	00000000	00000002	00000000	00000002	00000001	00000000	00000001	00000000
WriteDataM[31:0]	00000002	00000000	00000004	00000010	00000000	00000001	00000002	00000000	00000000	00000002	00000000	00000002	00000001	00000000	00000001	00000000
ForwardAE[1:0]	0	2	0	2	0	2	0	0	2	0	2	0	1	0	1	0
ForwardBE[1:0]	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
StallF	0															
StallD	0															
FlushE	1															

En este segmento intermedio, se observa el procesador ejecutando los bucles de cálculo. La señal ALUResultE presenta valores cambiantes que corresponden a las operaciones aritméticas de la multiplicación por software. Se destaca la activación de ForwardAE y ForwardBE con valores 1 o 2, lo cual confirma que el Hazard Unit detecta dependencias de datos y resuelve riesgos de tipo *Data Hazard* mediante adelantamiento (forwarding) desde las etapas MEM o WB directamente a la ALU. Esto permite que el procesador mantenga un flujo de ejecución constante sin insertar paradas (Stalls) innecesarias, a pesar de la alta presión sobre los registros.



Finalización del algoritmo y escritura de resultados (RV32C). En los ciclos finales de la simulación, el pipeline completa el cálculo de la Matriz C. La señal MemWrite habilita la escritura en la memoria de datos, mientras que DataAdr recorre las direcciones correspondientes a la Matriz C (base 0x20 en adelante). En WriteData, se observa el volcado de los resultados finales (4,4,a,8), cumpliendo con la lógica del algoritmo. Finalmente, al detectarse la escritura en la dirección 0xc8 (200 decimal) alrededor de los 1,480 ns, el testbench.v dispara la señal de finalización (\$finish), validando que el programa ha concluido exitosamente toda la secuencia de cálculos.



Como se puede observar en la imagen, en la RAM desde su registro 8 al 11 se puede observar el resultado de la nueva matriz. Esto sale igual al finalizar ambos códigos por lo que se puede confirmar que el algoritmo funciona correctamente tanto usando operaciones de 32 bits como de 16 (Híbrido).

3.2.3 Comparativa

Métrica Analizada	Versión RV32I Puro	Versión RV32C (Híbrida)	Beneficio
Tamaño del Programa (Code Footprint)	~ 276 bytes	~ 198 bytes	Reducción del 28.2% en ocupación de memoria de instrucciones.
Tiempo de Ejecución Total	3,900 ns	4,280 ns	Ligero aumento en el tiempo de ejecución (Trade-off de compresión).

Análisis de Resultados:

1. **Densidad de Código (El principal beneficio de RV32C):** Al igual que en el algoritmo de ordenamiento, la multiplicación de matrices evidencia una mejora sustancial en la densidad de código. El uso extensivo de `c.addi`, `c.slli` y las instrucciones de control de flujo compactas (`c.j`, `c.jal`, `c.beqz`) compactó masivamente los largos bloques de los bucles anidados, generando un ahorro cercano al 30% en la memoria de instrucciones.
2. **Penalización de Rendimiento (El trade-off arquitectónico):** Al carecer de multiplicador por hardware, la subrutina `soft_mul` somete al pipeline a una iteración constante. En la versión RV32C, la instrucción `c.beqz` obliga a forzar el segundo operando a `x0`, y el formato destructivo de las operaciones de ALU requirió inserciones adicionales de movimientos de registros para no perder las variables iteradoras. La simulación demostró que la Unidad de Riesgos maneja de forma impecable los vaciados (FlushE) repetitivos, logrando un control de flujo exacto. Sin embargo, la sobrecarga de saltos y la reestructuración de variables temporales para sortear las limitaciones de 16 bits incrementaron ligeramente el conteo total de instrucciones dinámicas, alargando la latencia de ejecución frente al código puro de 32 bits.

Conclusion:

Los resultados validan que la arquitectura segmentada maneja la extensión 'C' con total robustez. La penalización en el tiempo de ejecución es un compromiso (*trade-off*) aceptable, ya que la arquitectura logra su objetivo principal: maximizar la densidad del código, optimizando el espacio físico en sistemas embebidos.

4. Bibliografía

- I. Harris, D., & Harris, S. (2020). *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann.
- II. RISC-V Collaboration. (s.f.). riscv-gnu-toolchain: GNU toolchain for RISC-V, including GCC. GitHub. Recuperado el 23 de junio de 2026, de <https://github.com/riscv-collab/riscv-gnu-toolchain>